Software process models

Version 2.0

Motivation

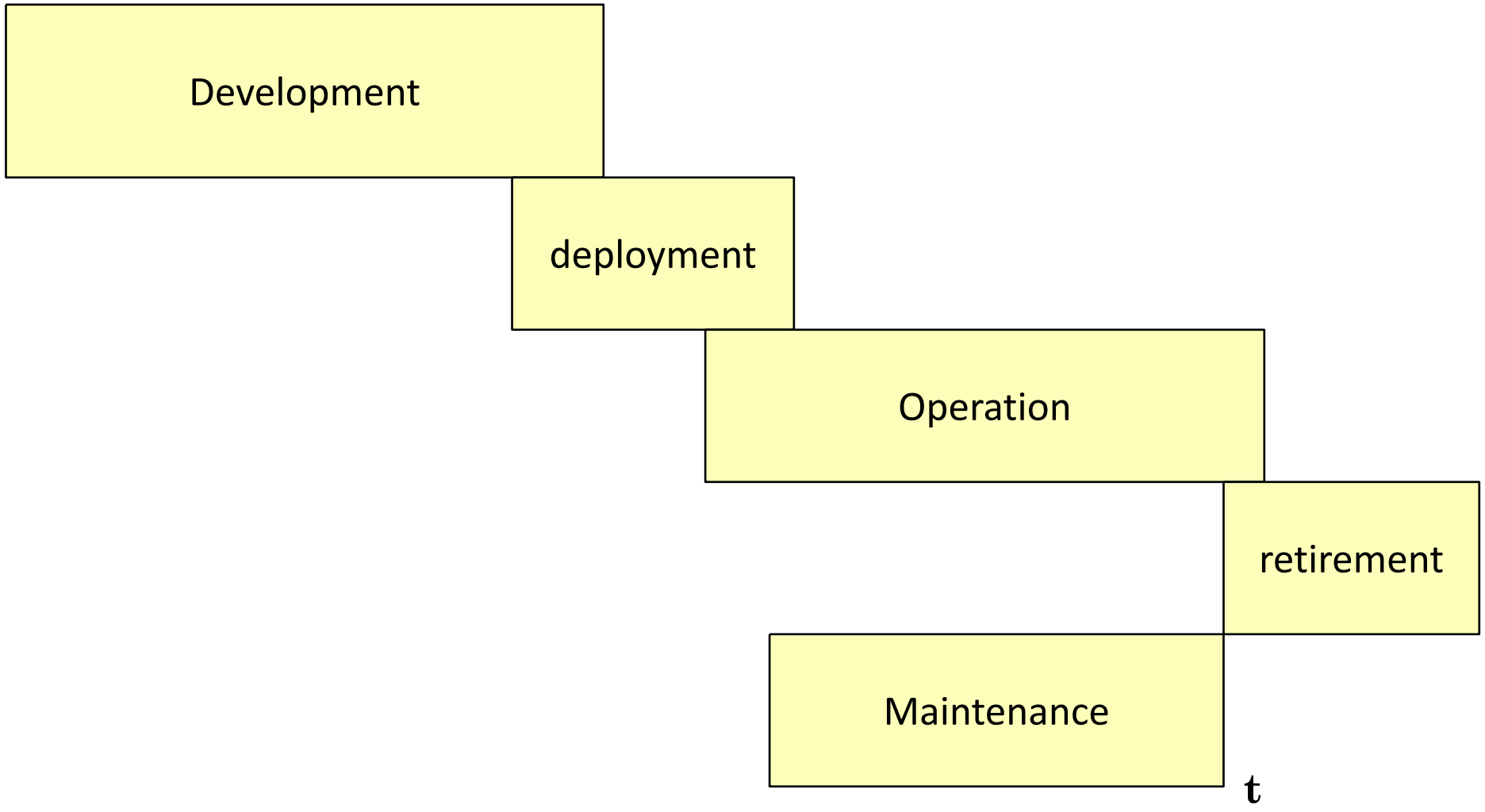
- We have analyzed activities
  - Requirement, design, ..
- And the techniques that can be used to perform them
  - UML modeling, prog languages, WB BB testing, ..
- But how to organize all?

Outline

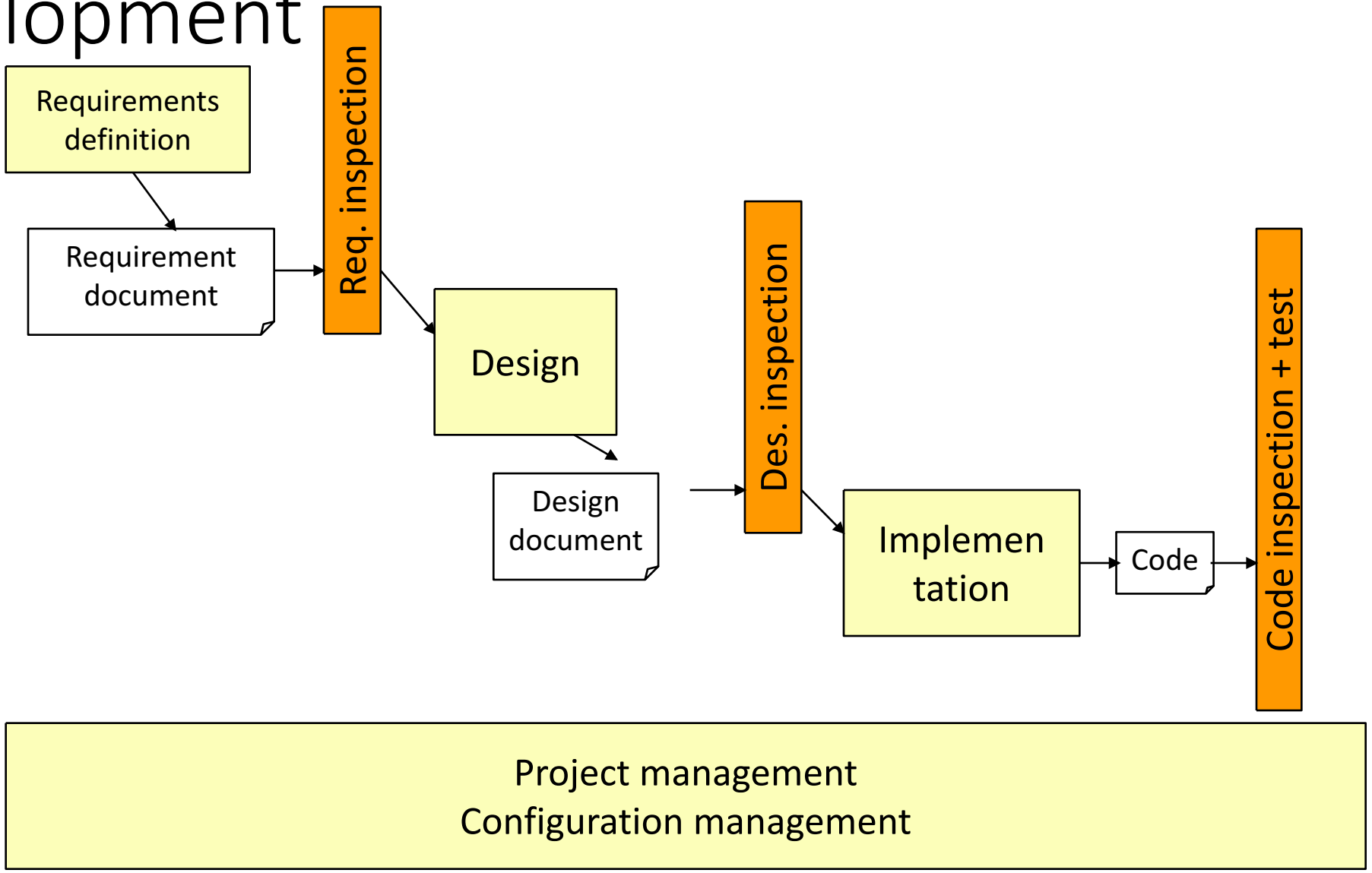
- Phases and Activities
- Processes, process models
- Projects
- Selection of process for project

Phases and activities

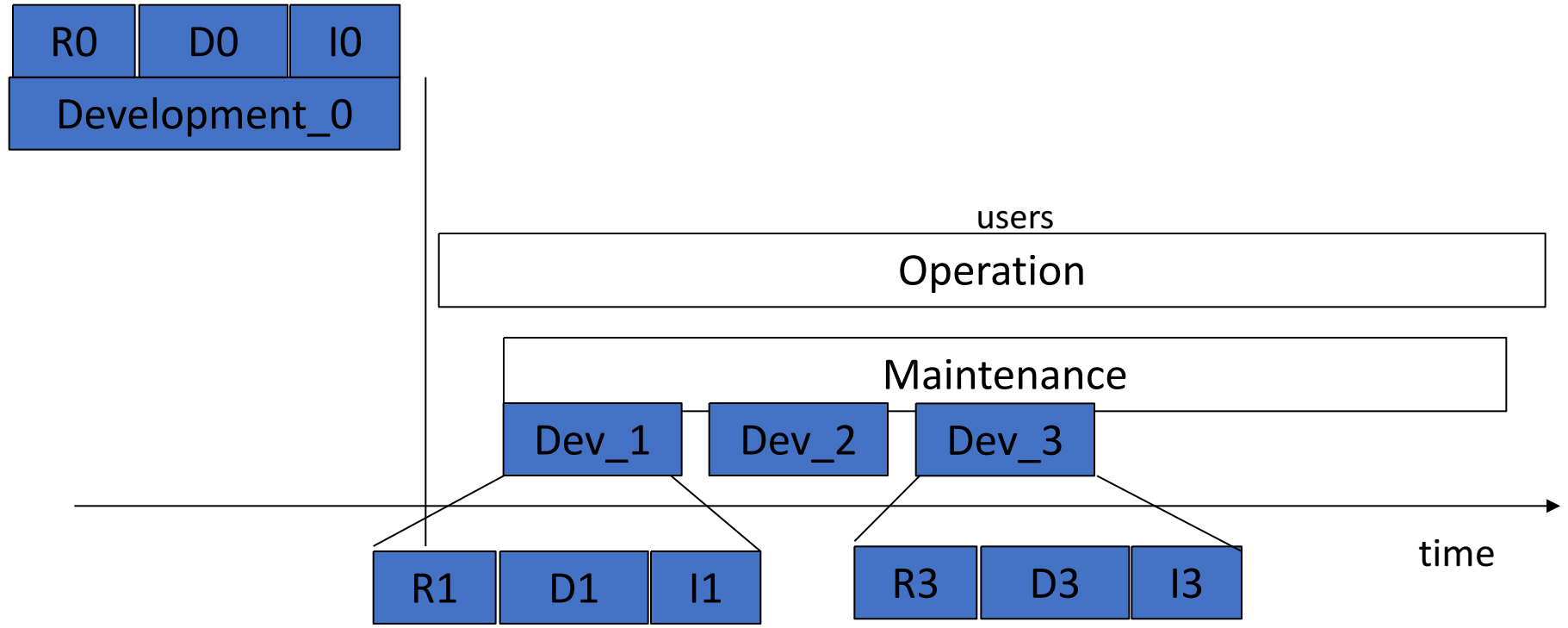
Main Phases



Development



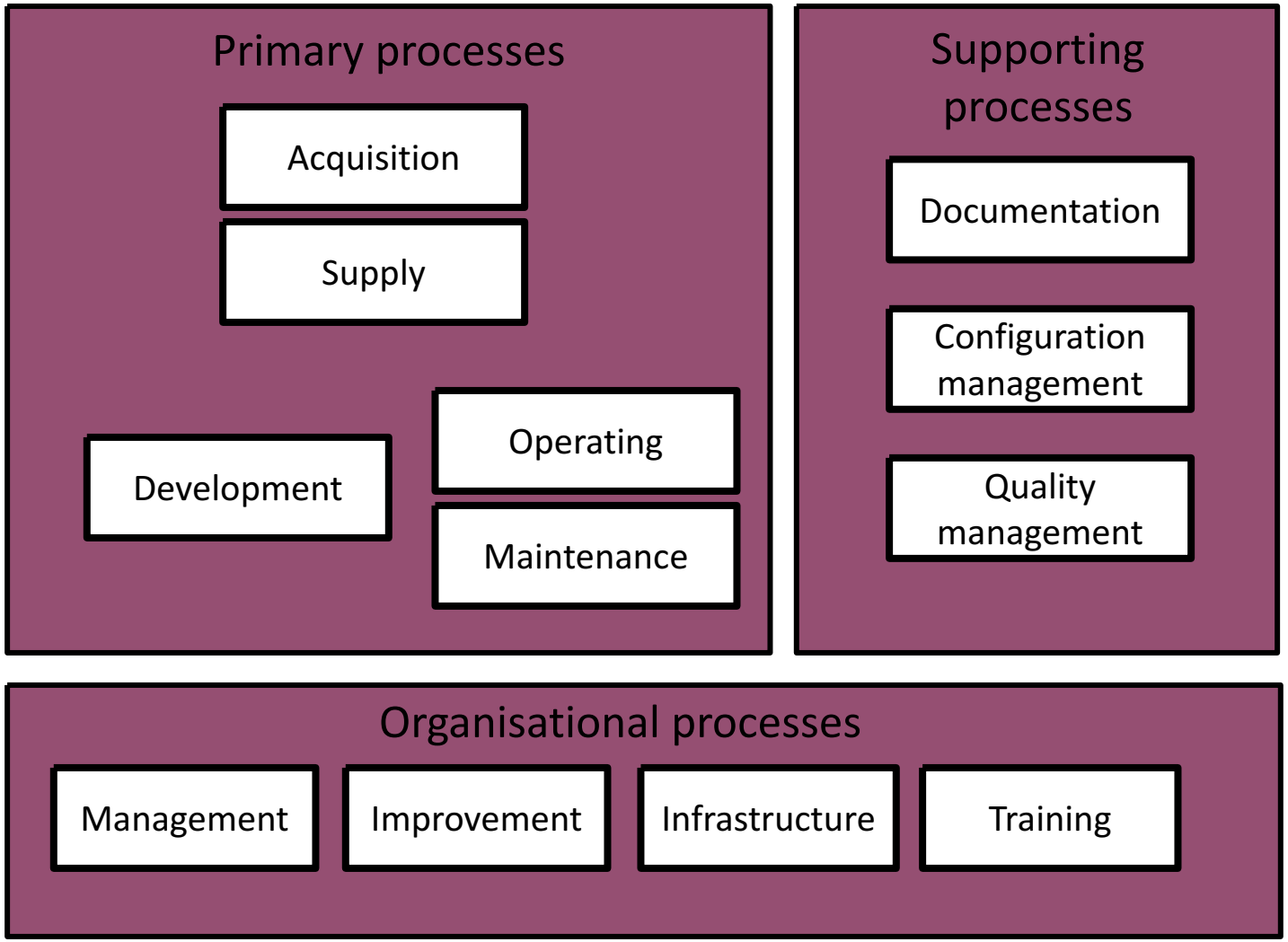
Maintenance



ISO/IEC 12207

- ISO == Int Standard Organization
  - Identifies processes
  - Identifies entities responsibles of processes
  - Identifies products of processes

ISO/IEC 12207



Primary processes

- Acquisition (manage suppliers)
- Supply (interaction with customer)
- Development (develop sw)
- Operation (deploy, operate service)
- Maintenance

Supporting

- Documentation of product
- Configuration management
- Quality assurance
  - Verification and Validation
  - Reviews with customer
  - Internal audits
  - Problem analysis and resolution

Organizational

- Project management
- Infrastructure management
  - Facilities, networks, tools
- Process monitoring and improvement
- Training

Ex. Software development

- Activity 5.3 Software development is decomposed in tasks
  - 5.3.1 Process Instantiation
  - 5.3.2 System requirements analysis
  - 5.3.3 System architecture definition
  - 5.3.4 Software requirements analysis
  - 5.3.5 Software architecture definition
  - 5.3.6 Software detailed design
  - 5.3.7 Coding and unit testing
  - 5.3.8 Integration of software units
  - 5.3.9 Software validation
  - 5.3.10 System integration
  - 5.3.11 System validation

V&V Tasks

- Coding and verification of components (5.3.7.)
- Integration of components (5.3.8.)
- Validation of software (5.3.9.)
- System Integration (5.3.10.)
- System validation (5.3.11.)

Subtasks

- Coding and verification of components (5.3.7.)
  - Definition of test data and test procedures (5.3.7.1.)
  - Execute and document tests (5.3.7.2.)
  - Update documents, plan integration tests (5.3.7.4.)
  - Evaluate tests (5.3.7.5.)
- Integration of components (5.3.8.)
  - Definition of integration test plan (5.3.8.1.)
  - Execute and document tests (5.3.8.2.)
  - Update documents, plan validation tests(5.3.8.4.)
  - Evaluate tests (5.3.8.5.)

ISO 12207

- Only list of activities
- Independent of process model
  - Waterfall, iterative, ..
- Independent of technology
- Independent of application domain
- Independent of documentation

Process models

How to organize activities?

- What?
  - Activities, documents
- Who does what?
  - Roles and responsibilities
- When?
  - Temporal constraints between activities
- Under constraints
  - Laws, standards

Software process

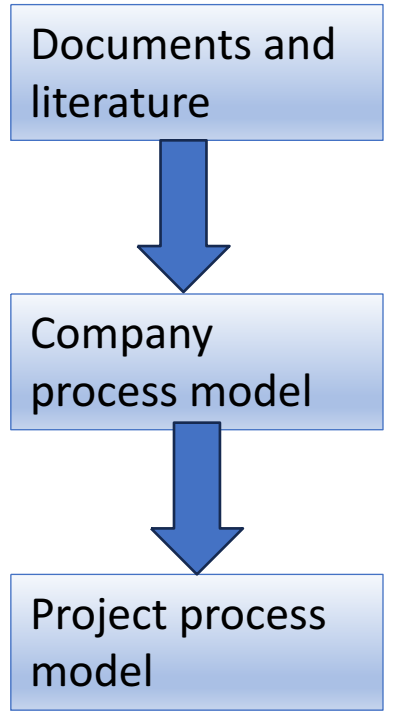
- Activities
  - Requirement, design, ..
- Products
  - Requirement document, design document ..
- Roles
  - Project manager, analyst, tester, ..
- Process model
  - Temporal constraints on how activities should be executed, responsibilities

Process models

- From standards / documents
  - ISO 15288 (system engineering activities)
  - ISO 12207 (sw engineering activities)
  - ISO 9001, ISO 9000-3
  - CMM-I
- From literature
  - Waterfall, RUP, Agile,

Mature company approach

- Company process model
  - Suggested / required activities, documents, milestones
- Project process model
  - (see 5.3.1 Process Instantiation in 12207)
  - Project defines specific model to follow, in function of criticality, cost, size, technology and application domain
  - Quality team reviews the decision



Process instantiation

- Criticality (safety critical, mission critical, none) and size are the key factors to select a process

- Ex manned mission to Mars vs. Candy Crush, radically different processes

Key question

- How to define or select the most suitable process for a certain project?

Process conformance

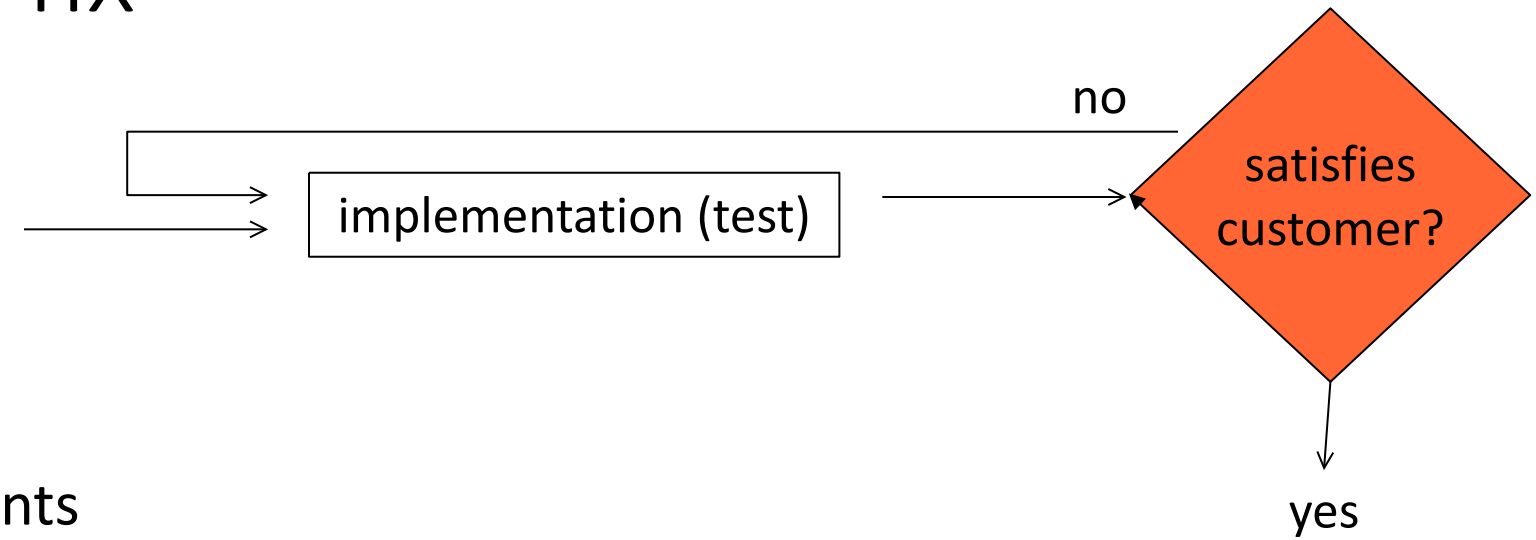
- Consistency between
  - Actual process followed in a project
  - Process model defined in documents

In theory, theory and practice are the same

..

In practice, they are not

Build and fix



- No requirements
- No design
- No validation of requirements/design

- May be ok for solo programming
- Does not scale up for larger projects

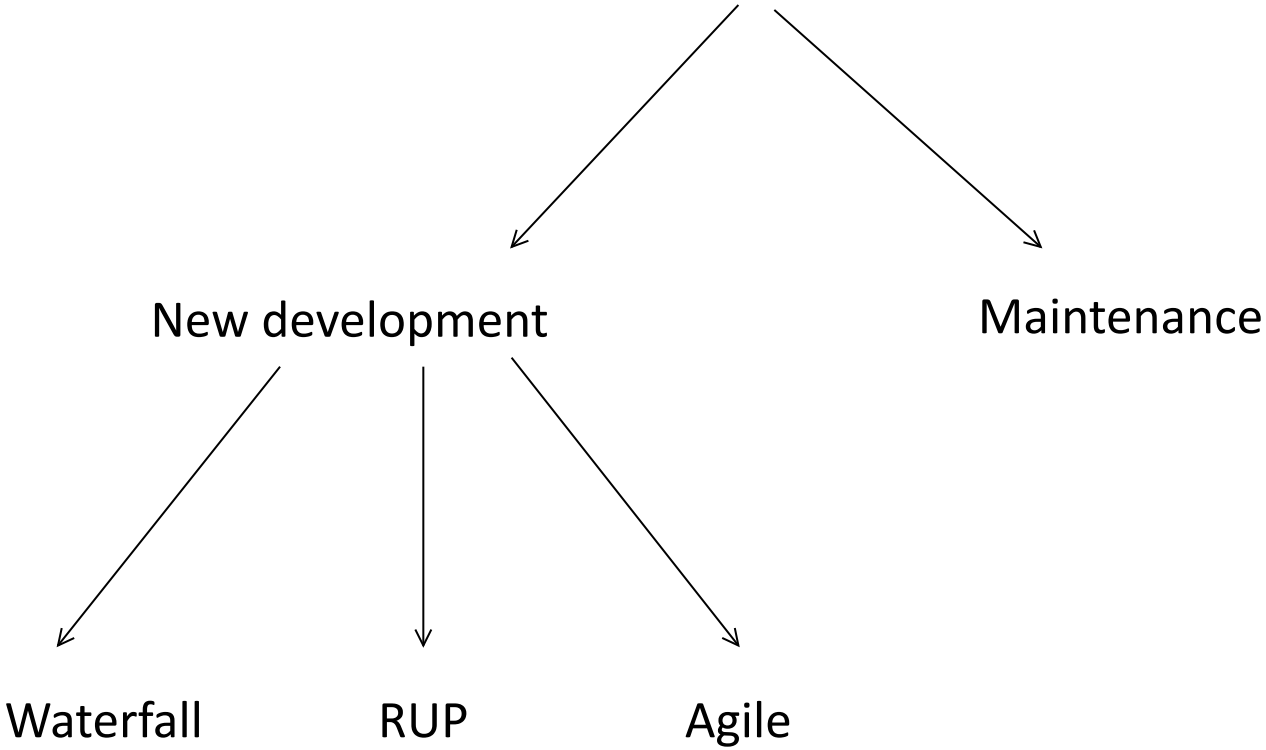
Models

- Main constraints in defining a process model
  - New development vs maintenance
  - Compliance to standards, laws
    - Correlates with criticality: safety critical, mission critical, no critical
  - Size of end product
    - Correlates with effort, calendar duration, size of staff involved, number and geographical distribution of teams

Model dimensions

- Main differences in process models
  - New development vs maintenance
  - Sequential vs parallel activities
  - Iterations (a long one vs many short ones)
  - Emphasis on documents (yes vs no)

Models



Models, new development

- Waterfall and variants
  - Waterfall, waterfall + prototype
  - Incremental
- RUP
- Agile

- Reuse
  - as modifier in all models

Waterfall

	Waterfall
Sequential parallel activities	Sequential
Iterations	One, long
Emphasis on documents	yes

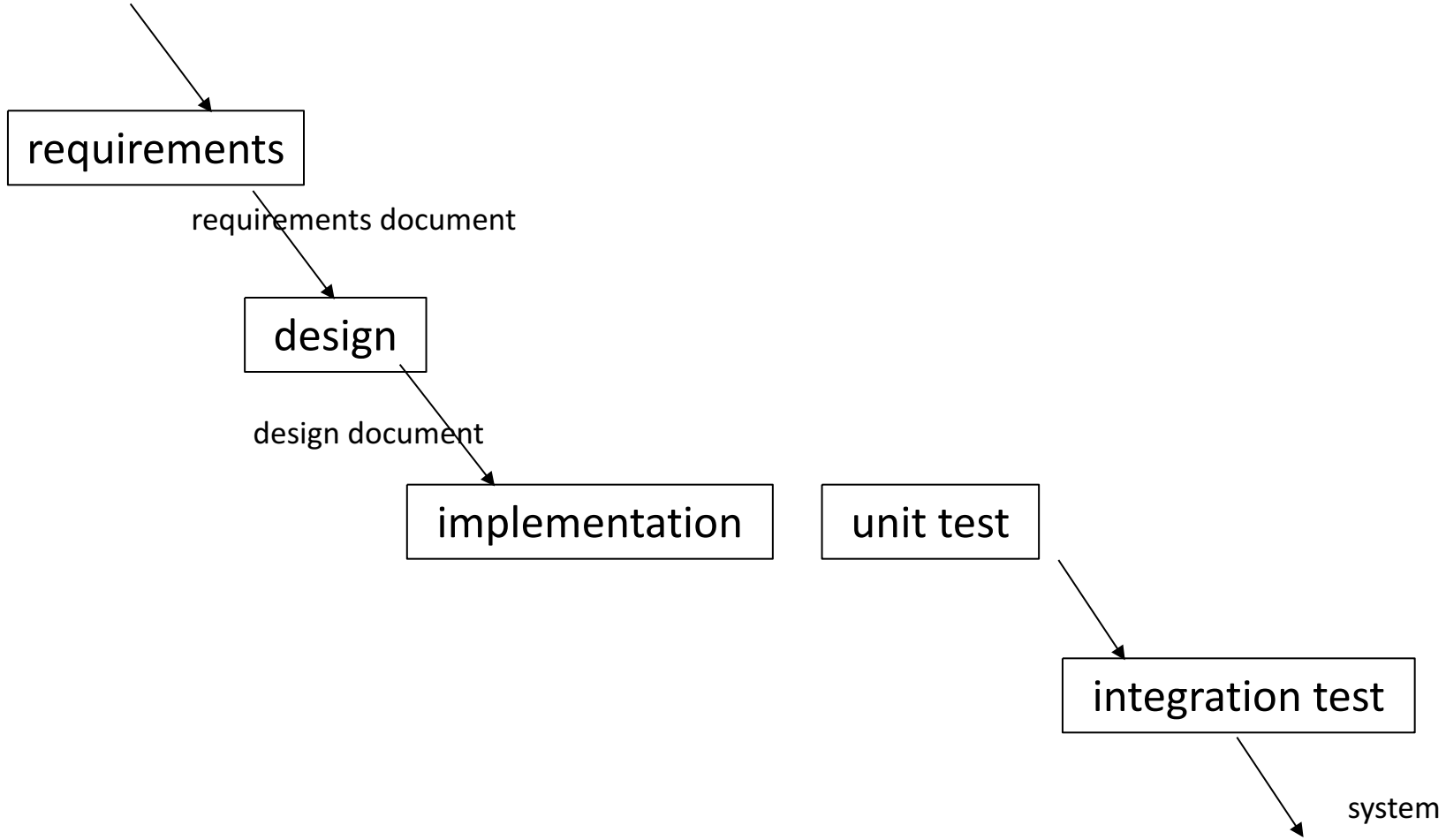
Waterfall

- [Royce1970]
- Sequential activities
  - Activity produces document/deliverable
  - Next activity starts when previous is over and freezes the deliverable
- Document driven

Waterfall

- (One) long iteration
  - Ideal, never feasible in practice
  - Change to a document implies re-doing all activities depending on it
    - Change to requirement → redo all activities
    - Change to design → redo all except requirements
    - Etc
  - Changes are long and expensive

Waterfall



Problems

- Lack of flexibility
  - Rigid sequentiality
  - Change to a document has heavy impact
    - tension to avoid changes
    - worst impact changes are to requirements and design
      - that normally are the most faulty
      - that should change to follow changes in context/customer needs
- Burocratic

Suitability

	Waterfall
New development / maintenance	Mostly new developments
Compliance	Yes, documents
Size	Large projects, distributed teams, distributed contractors

Examples

- Automotive, aerospace
  - Thousands of components
  - Hundreds of subcontractors to OEM
- Development cycles
  - Car: 2-3 years
  - Airplane: 3-7 years

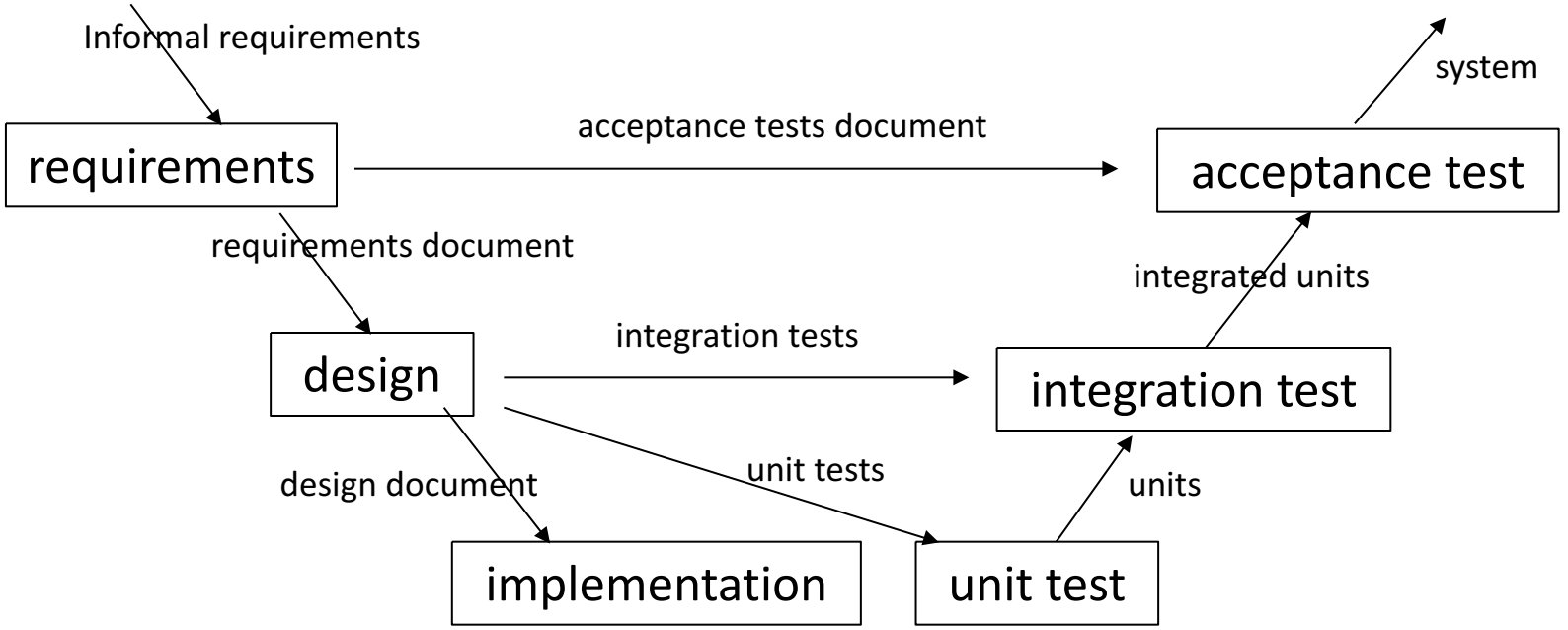
Variants of waterfall

- V Model
  - ISO 26262, IEC 61508
- Waterfall + prototype
- Incremental

V Model

- Similar to waterfall
- Emphasis on VV activities
- Acceptance tests written after/with requirements
- Unit/integration tests written after/during design

V Model



ISO 26262, IEC 61508

- As examples of system process model, waterfall / V model

ISO 26262

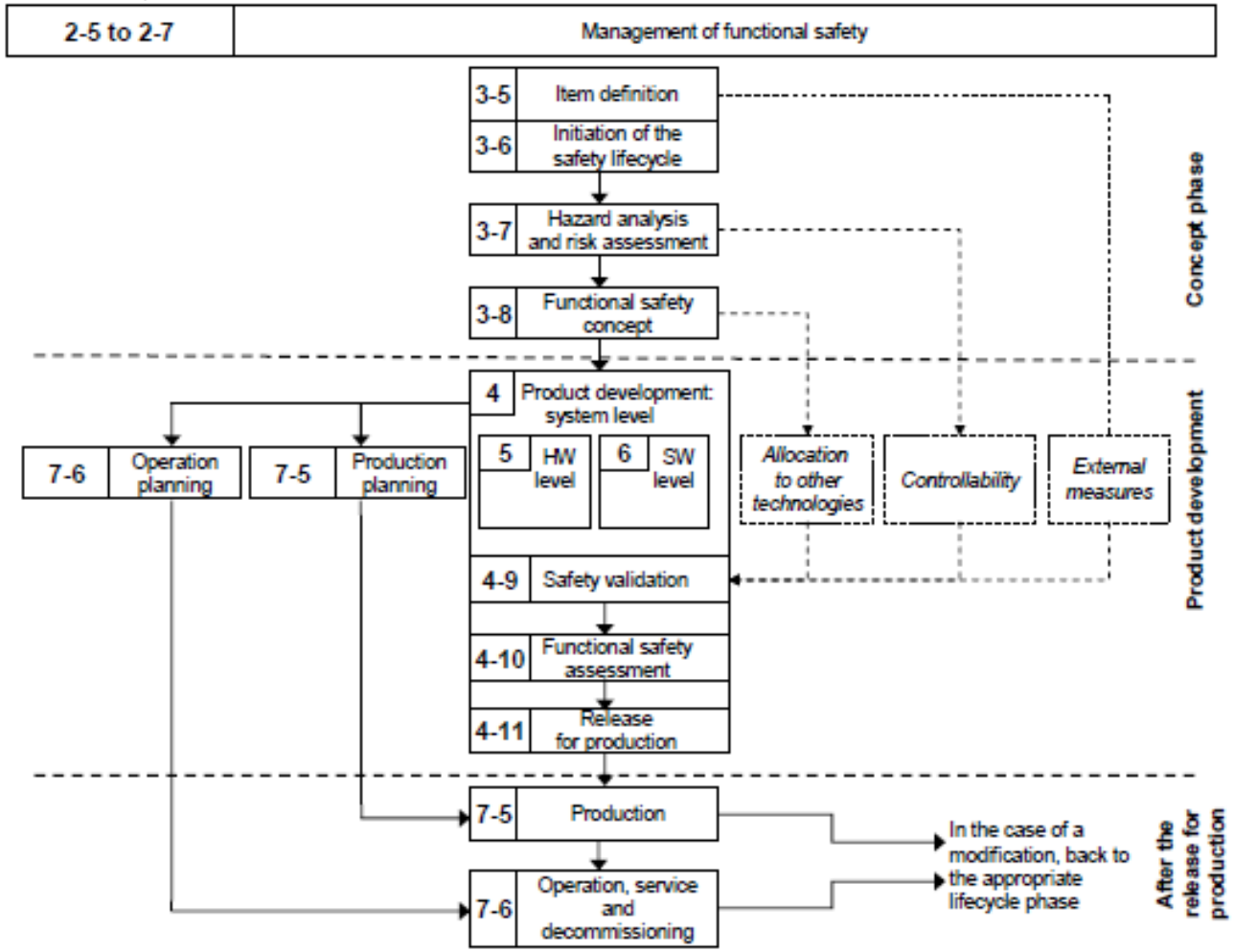
- Road Vehicles, functional safety
- Adaptation of IEC 61508
  - Functional safety for systems with EEPs (computers)

  - SIL – Safety Integrity Level
  - ASIL – automotive SIL, A (light red) B C D (red)

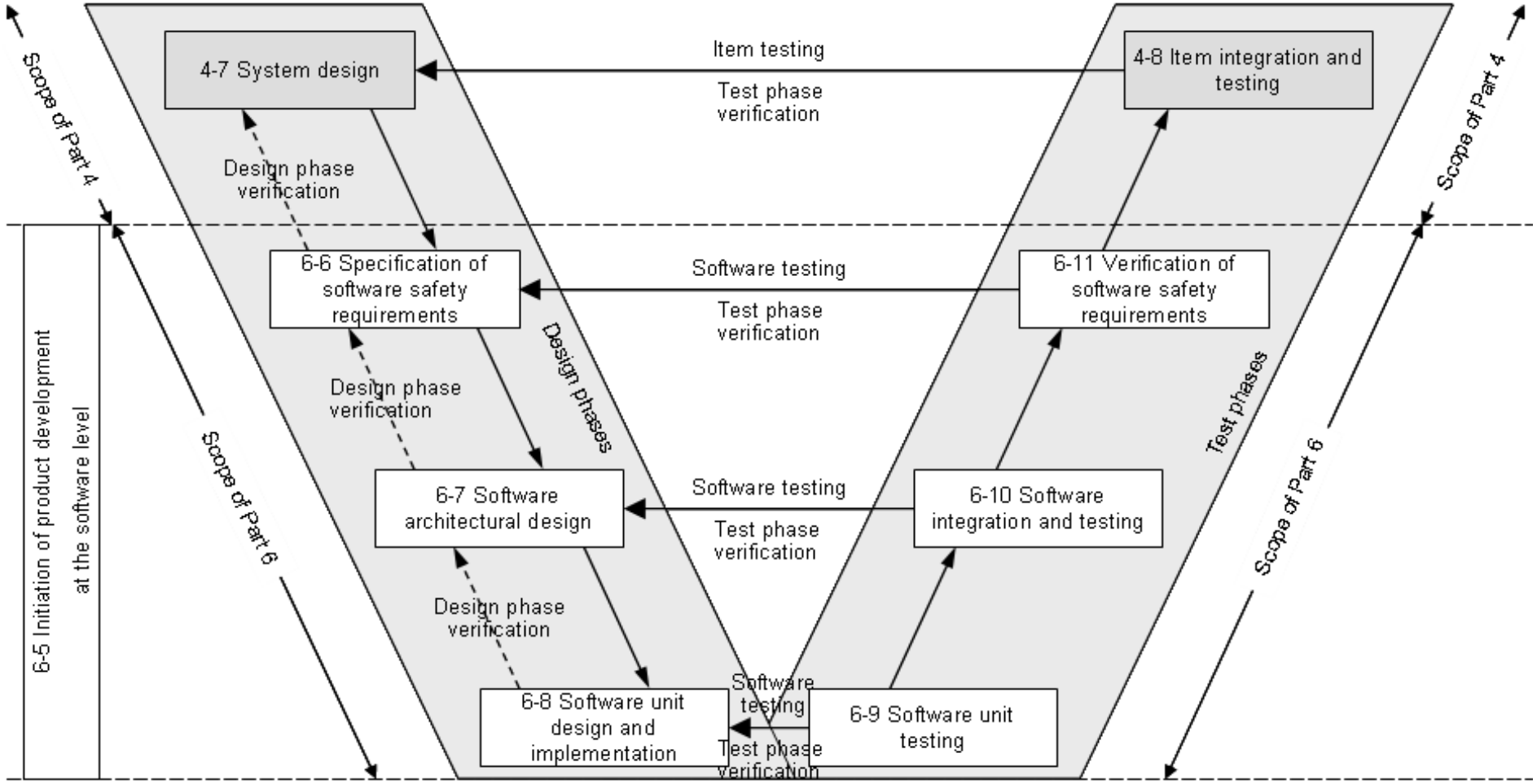
Structure

- Vocabulary
- Management of functional safety
- Concept phase
- Product development at the system level
- Product development at the hardware level
- Product development at the software level
- Production and operation
- Supporting processes
- Automotive safety integrity levels

Safety Lifecycle



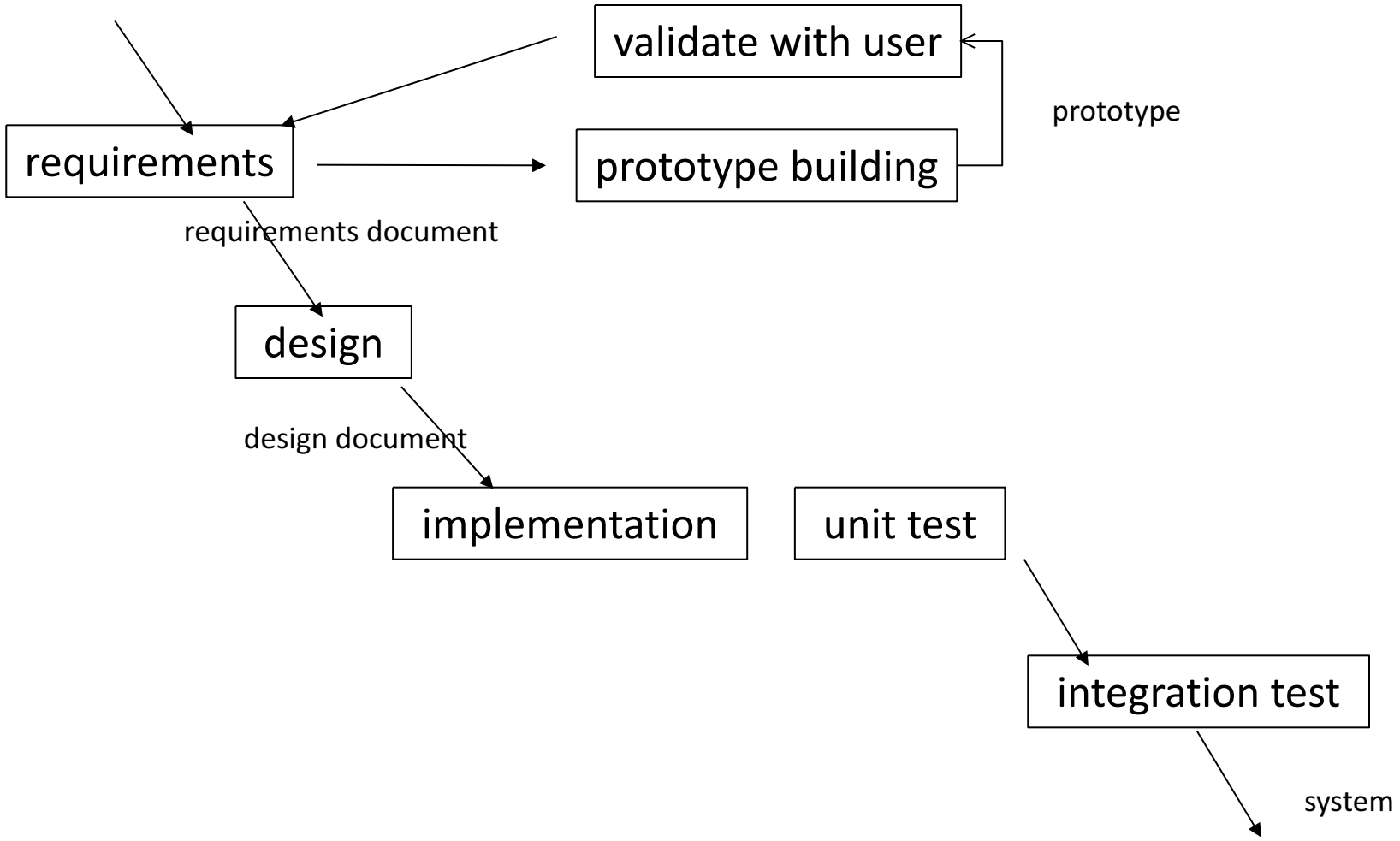
Software lifecycle



Prototyping + waterfall

- Quick
  and dirty prototype to validate/analyze requirements
- Then same as waterfall

Prototyping + waterfall



Prototyping

- Advantages
  - Clarify requirements
- Problems
  - Requires specific skills to build prototype (prototyping language)
  - Business pressures to keep prototype (when successful) as final deliverable

Prototype in sw

- Less functions
- Other platform
  - Final product in C, embedded
  - Prototype in Java, on PC
  - Or
  - Prototype in Matlab

Prototyping

- The idea can be applied to other parts of a project
  - GUI prototyping (see requirements chapter)
  - Design prototyping
  - Performance

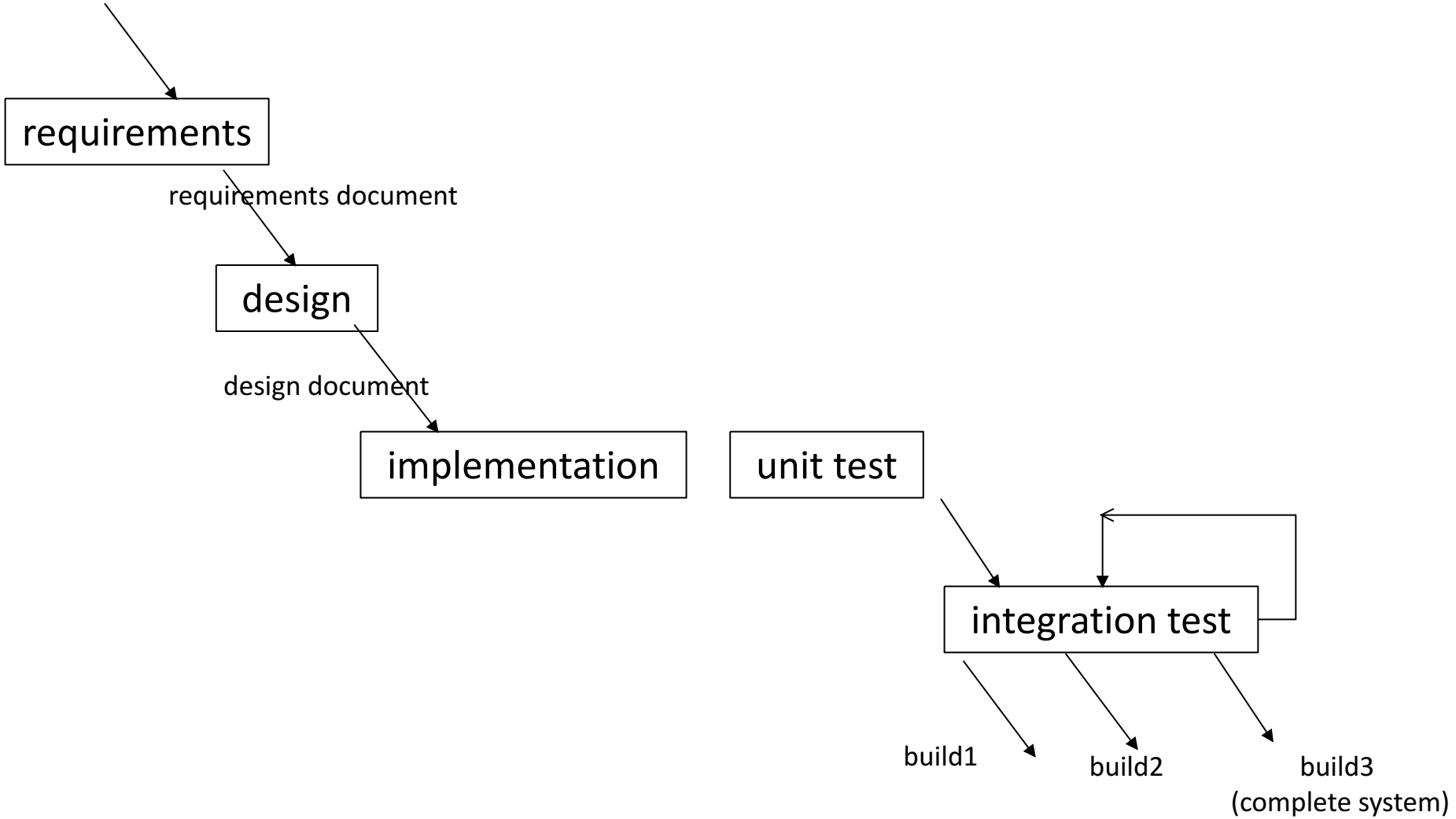
Incremental

- In Waterfall, integration activity produces the complete system, that is delivered to customer in one shot
- Incremental is same as waterfall, but integration is split, every loop in integration produces a part of the system (the system is delivered in several increments)
  - One increment == ‘build’

Incremental

- Pros
  - Earlier feedback from user / customer
  - Increments that depend on external components can be delayed

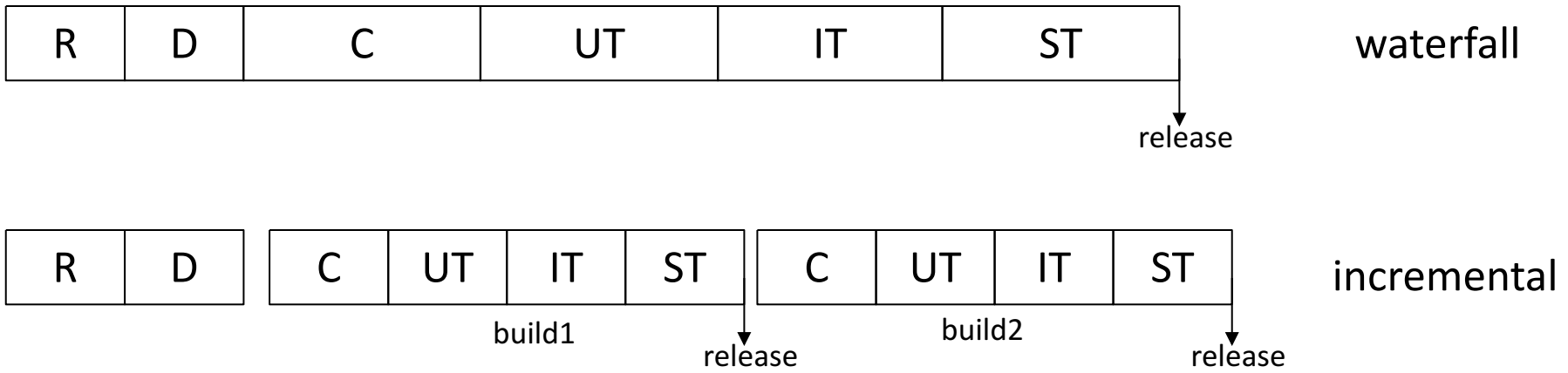
Incremental



Incremental

- Requirements: R1, R2, R3
- Architecture: C1, C2, C3, C4
- Planning: 3 iterations
- Iteration1
  - R1, requires C1, C2
  - Develop and integrate C1, C2,
  - Deliver R1
- Iteration2
  - R2, requires C1, C3
  - Develop C3, integrate C1, C2, C3
  - Deliver R1 + R2
- Iteration3
  - R3, requires C3, C4
  - Develop C4, integrate C1, C2, C3, C4
  - Deliver R1 + R2 + R3

Comparison

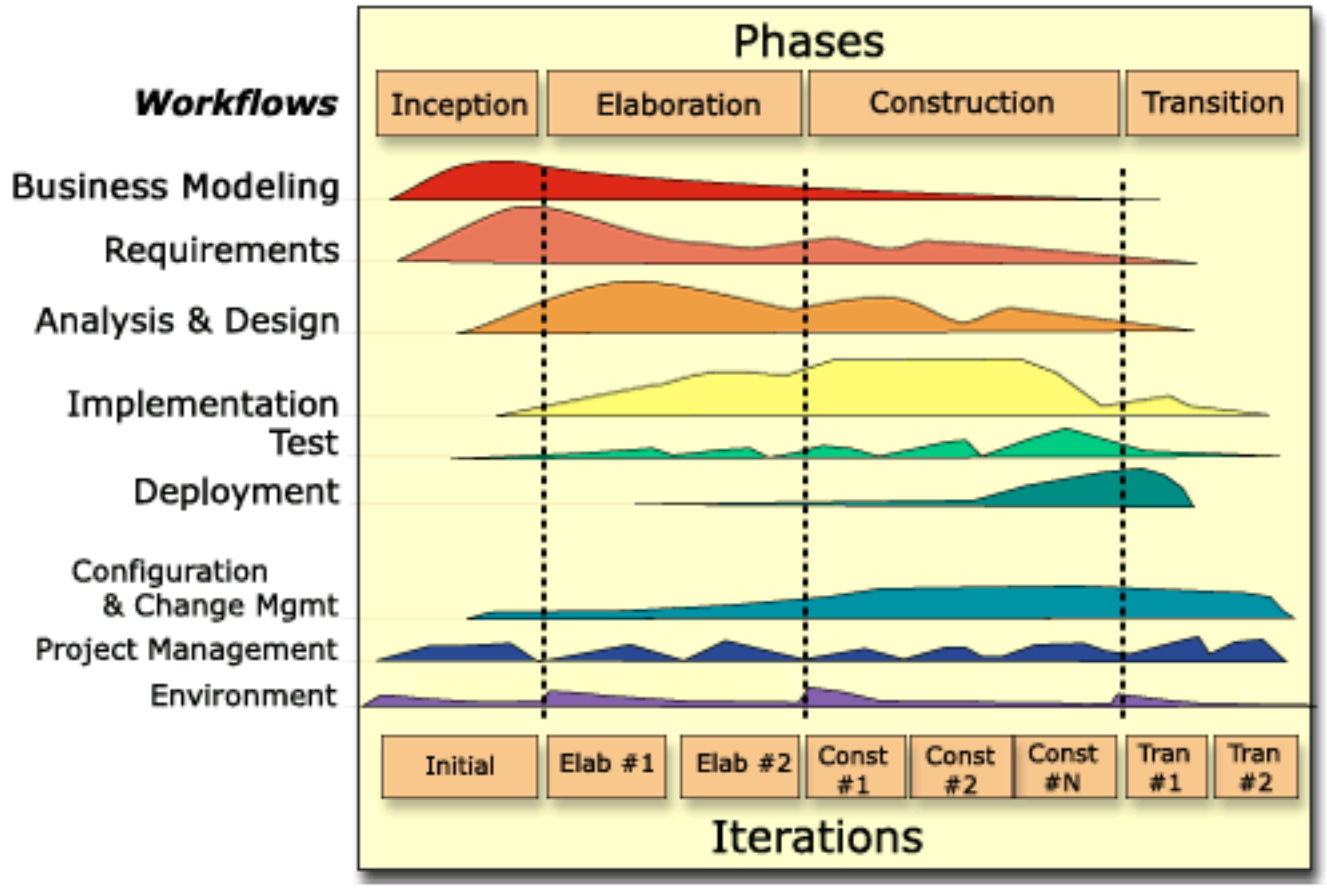


Legenda:R= Requirements, D= Design, C=Coding, UT=Unit Test,
IT= Integration Test, ST = System Test

Rational Unified Process (RUP)

	RUP
Sequential /parallel activities	parallel
Iterations	One, long with sub iterations
Emphasis on documents	partial

(R)UP



Time sheet - sequential

Week	requirement engineering	design	coding	unit testing	integration testing	acceptan testing
apr 13 - 19	8					
apr 20 - 27		4				
apr 28 - 3		6				
may 4 - 10			25			
may 11 - 17			25			

Time sheet – parallel

Week	requirement engineering	design	coding	unit testing	integration testing	acceptance testing	management	git maven
apr 13 - 19	40	25					15	5
apr 20 - 27		20					5	
apr 28 - 3			8	6			6	
may 4 - 10			7	8			4	
may 11 - 17	1	3	2	5			11	
may 18 - 24								

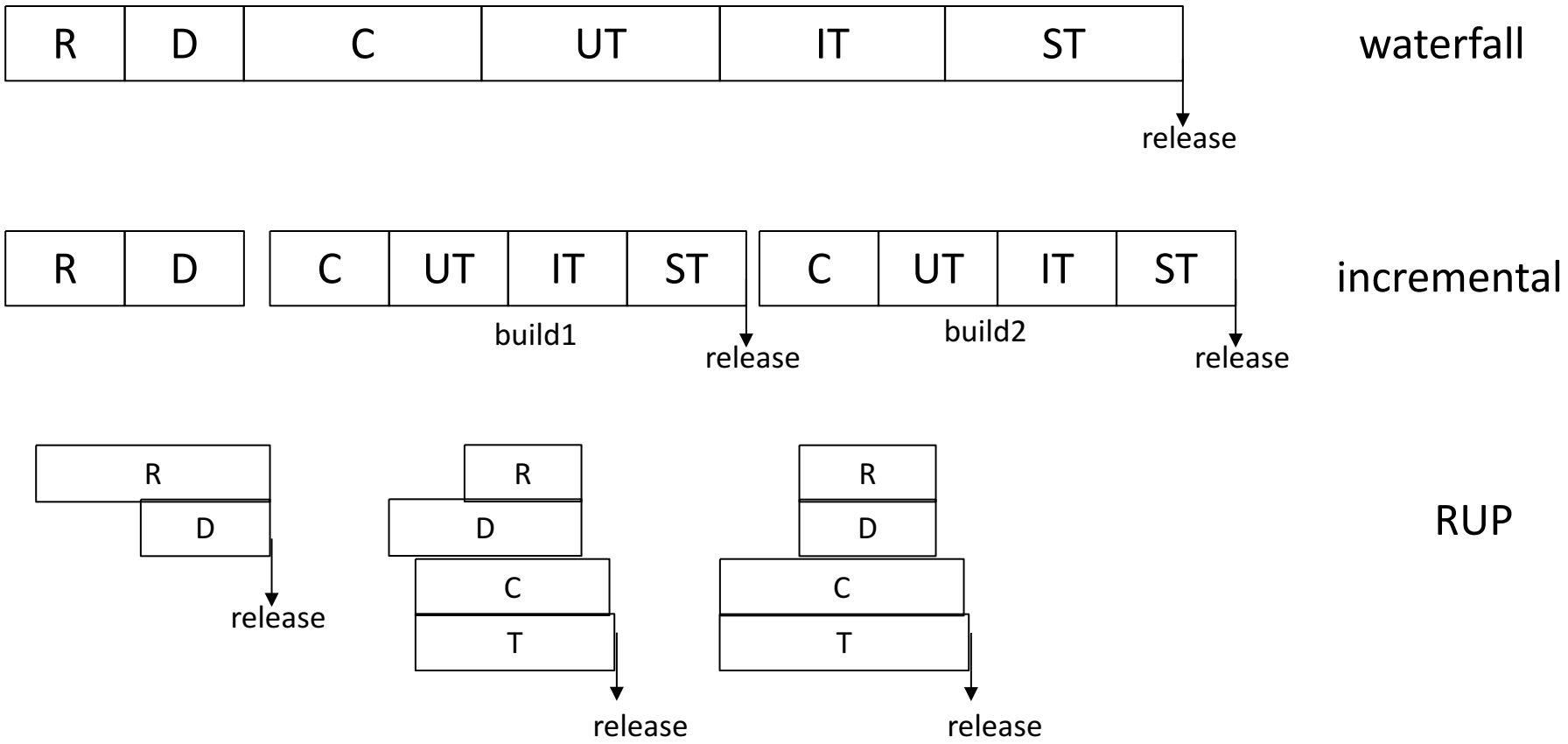
(Rational) Unified Process

- Proposed in 1999 by
  - Grady Booch
  - Ivar Jacobson
  - James Rumbaugh
- Characteristics
  - Based on architecture
  - Iterative incremental

UP Phases

- Inception
  - Feasibility study; risk analysis; essential requirements; prototyping (not mandatory)
- Elaboration
  - Requirement analysis; risk analysis; architecture definition; project plan
- Construction
  - analysis, design, implementation, testing
- Transition
  - Beta testing, performance tuning; documentation; training, user manuals; packaging for shipment

Comparison



Legenda:R= Requirements, D= Design, C=Coding, UT=Unit Test,
IT= Integration Test, ST = System Test

Suitability

	RUP
New development / maintenance	Mostly new developments
Compliance	partial
Size	Also large projects, distributed teams

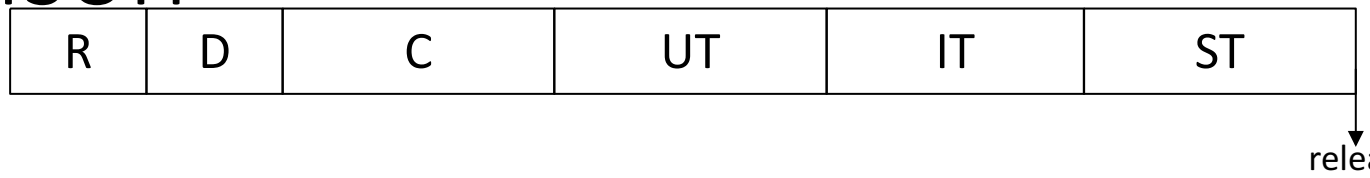
Agile

- Scrum
- XP
  - See other chapter

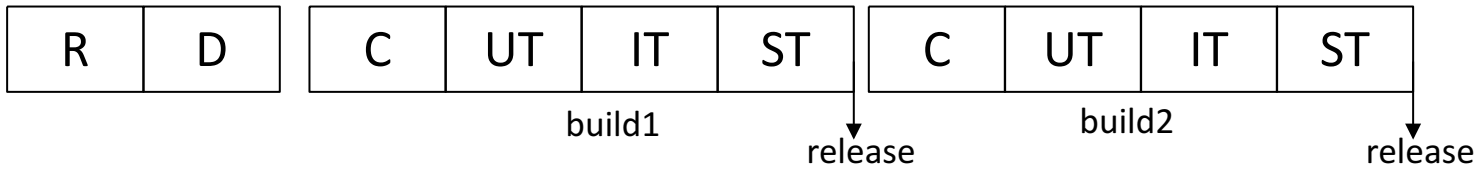
Agile

- Very skinny requirements and design
  - Requirements can change at every iteration
- Code and automated test cases
- Continuous integration
  - Every day, from day 0
- Iterations: 4 weeks

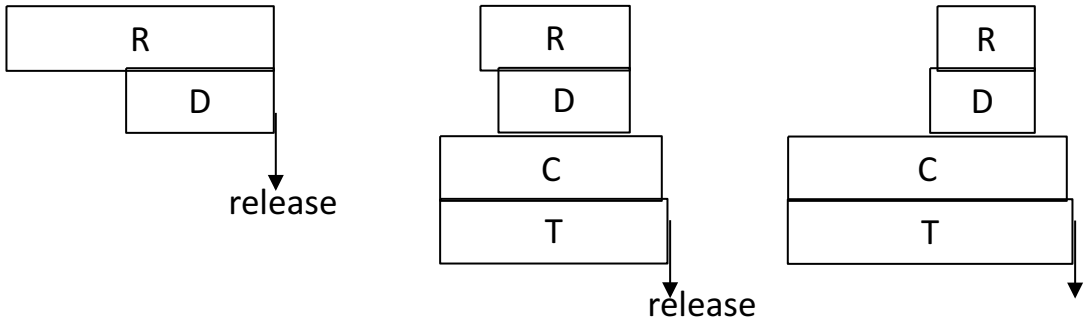
Comparison



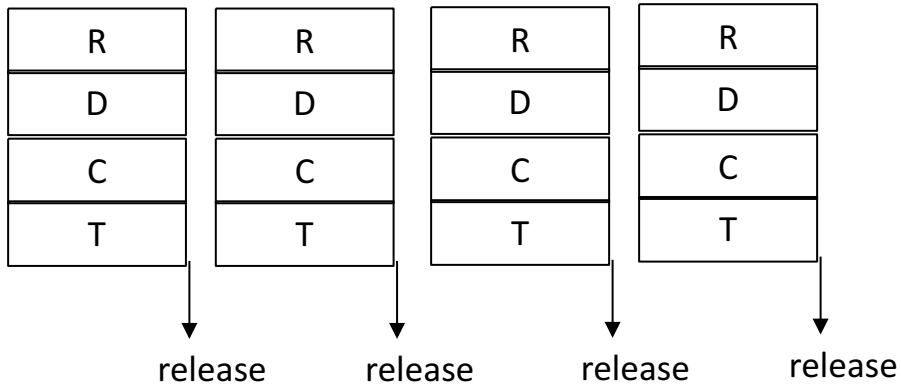
waterfall



incremental



RUP



Agile

Agile

	Agile
Sequential parallel activities	parallel
Iterations	Many, short
Emphasis on documents	no

Suitability

	Agile
New development / maintenance	both
Compliance	no
Size	Small projects, co located teams

Process options vs models

	Waterfall	RUP	Agile
Sequential parallel activities	sequential	parallel	parallel
Iterations	One, long	One, long with sub iterations	Many, short
Emphasis on documents	heavy	mild	no

Suitability

	Waterfall	RUP	Agile
New development / maintenance	Mostly new developments	Mostly new developments	both
Compliance	yes	partial	no
Size	Large projects, distributed teams and contractors	Also large projects, distributed teams	Small projects, colocated teams

Reuse

- Most projects reuse components
  - open/closed source
  - free/not free
- Pros
  - Immediate availability
  - Often higher quality and lower cost
- Cons
  - Ownership of source
  - Trade offs needed
  - Loss of control on evolution

Reuse - process

- Requirements
  - Must consider what is (un)available from components, and change requirements accordingly
- Design
  - Must evaluate and select components
  - Must consider constraints / issues from components and adapt design

Reuse – ex tradeoff in req

- R1, in an accounting package invoices should be produced in pdf, png, jpg

	No reuse	Component 1	Component 2
R1	Pdf, png, jpg	Yes pdf, jpg No png	Yes pdf, png No jpg
Cost	50	10	12
Time	3 months	1 month	1 month

Reuse - Activities

- Search and analysis of components
- Adaptation of requirements
  - Trade off between requirements and available components
- Design
- Implementation
  - ‘Glue’ software to integrate components

New development process models and Project management

- The process model chosen has deep impact on project management, and on the relationship customer - vendor

- Overall waterfall is naturally attached to a fixed price model, agile to a time and material model

	Fixed price	Time and material
	Customer and developer agree on price and duration of project, based on a detailed req doc	Customer pays to developer the amount of time (effort) and material used by developer
Requirements	Defined in detail, upfront.	Defined loosely
Requirement change	Discouraged. Requires change to contract (price and duration)	Possible
price	Defined and agreed before project start	Computed at project end
duration	Defined and agreed before project start	Defined at project end

Estimation

Waterfall	Agile
Requirements → effort Effort → cost	Requirements (very high level) → number of iterations Iteration → effort → cost

Contract

Waterfall – fixed price (and duration)	Agile - time and material
Legal part of contract + Technical annex to contract (contains rigid description of result of project == requirements)	Requirements, by definition of agile, can change
Goal of both parties is to stick to contract Variations in requirements may need renegotiation of contract	Contract can only describe agreed cost of personnel (cost of one iteration)

Maintenance process

Change

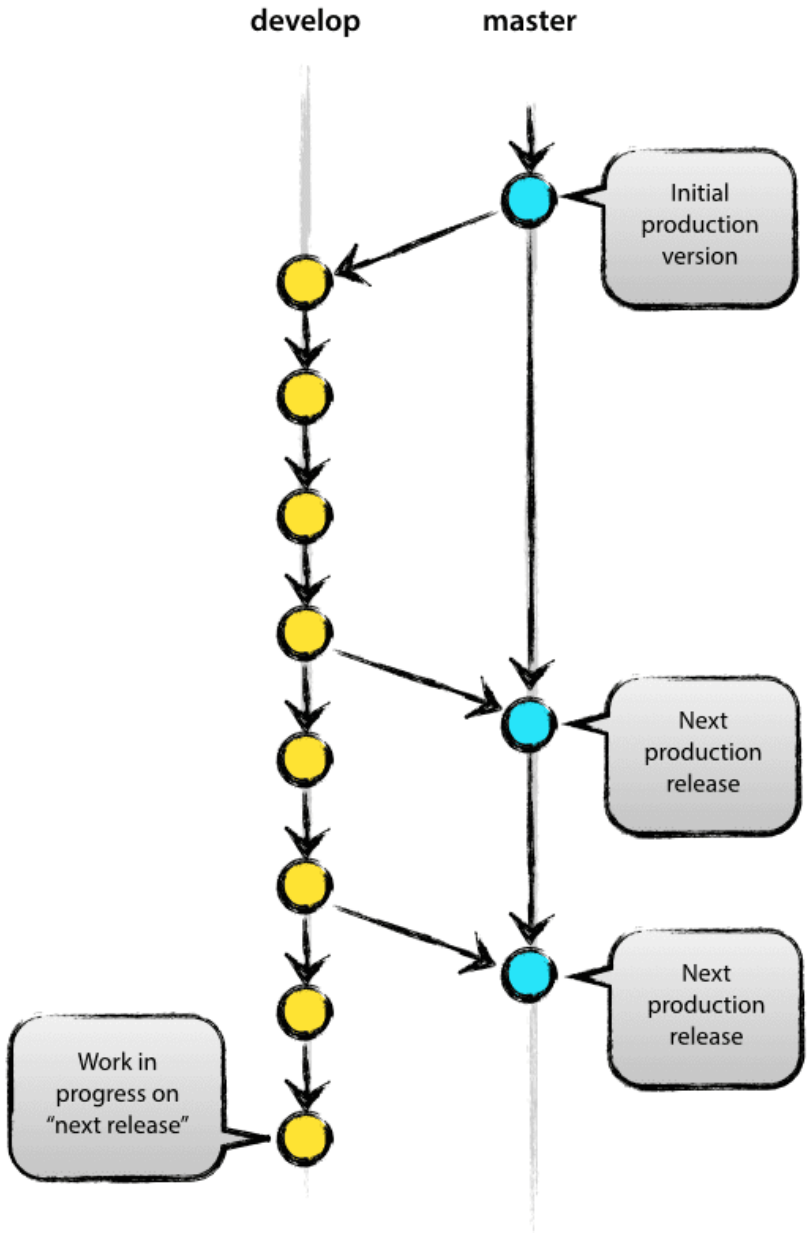
- The key element in a maintenance process
- Can be
  - A defect to be fixed (corrective maintenance)
  - A modification to an existing function / characteristic (perfective maintenance)
  - A new function / characteristic (evolutive maintenance, enhancements)

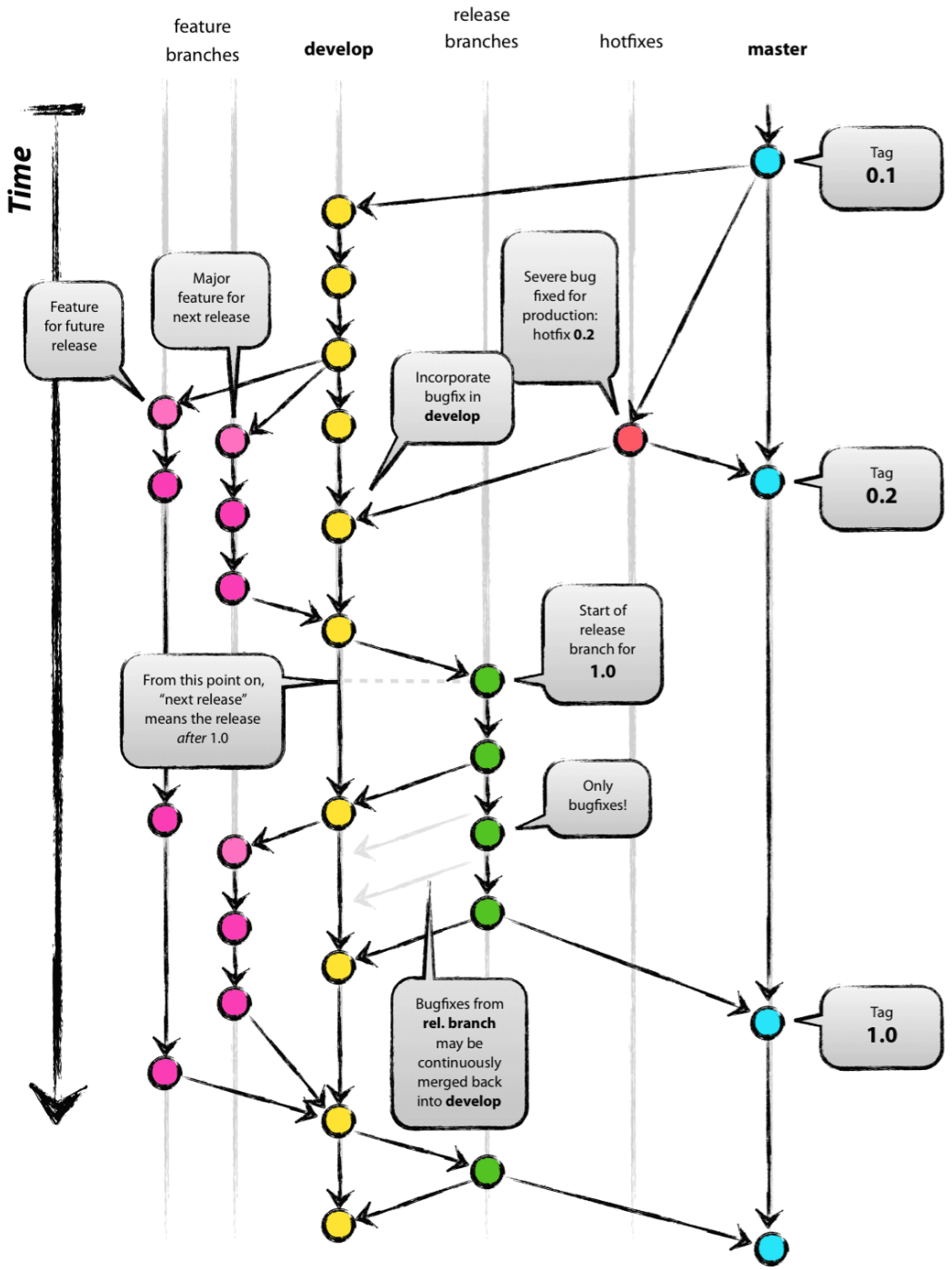
Change

- Can be originated by end users or developers
- Are received and processed by a group of maintainers

Product evolution

- The product evolves by applying a flow of changes
  - Tipically with regular releases
    - Major: every few months (ex 2 months)
    - Minor/critical: when needed
- Stable baseline (master)
  - Released regularly
- Working baseline (develop)
  - Where the maintainers work


develop
master
Initial production version
Next production release
Next production release
Work in progress on "next release"


feature branches
develop
release branches
hotfixes
master
Time
Feature for future release
Major feature for next release
Severe bug fixed for production: hotfix 0.2
Incorporate bugfix in develop
Tag 0.1
Tag 0.2
Start of release branch for 1.0
From this point on, "next release" means the release after 1.0
Only bugfixes!
Bugfixes from rel. branch may be continuously merged back into develop
Tag 1.0

Process

- Receive change request (CR)
- Filter
  - merge same/similar CR
  - discard unfeasible, incorrect CR
- Assess
  - Evaluate impact of CR (effort/cost, feasibility, impact on architecture and/or functionality)
  - Rank CR (using severity – for corrective CR, and importance – for evolutive CR)

Process (2)

- Assign CR to a (team of) maintainer(s), picking from ranked CRs
- Implement CR
  - Design, code, unit test, integration test
    - Maintainer
  - System test
    - Quality group
  - Insert in next release

Issues

- The flow of changes can radically change a product over time (years)
  - Need to control / mantain architecture (architecture erosion)
  - Need to control suitability for market /users
- Only a part of CRs (even corrective) are implemented

Roles: Service desk (help desk)

- Single entry point for CRs
  - End users directly write CRs on the tool (ex Git Issues)
    - Or
  - End users report issues to call center, call center operator enters CRs on tool
    - Or
  - Automated system to collect failures (cfr Windows OS)

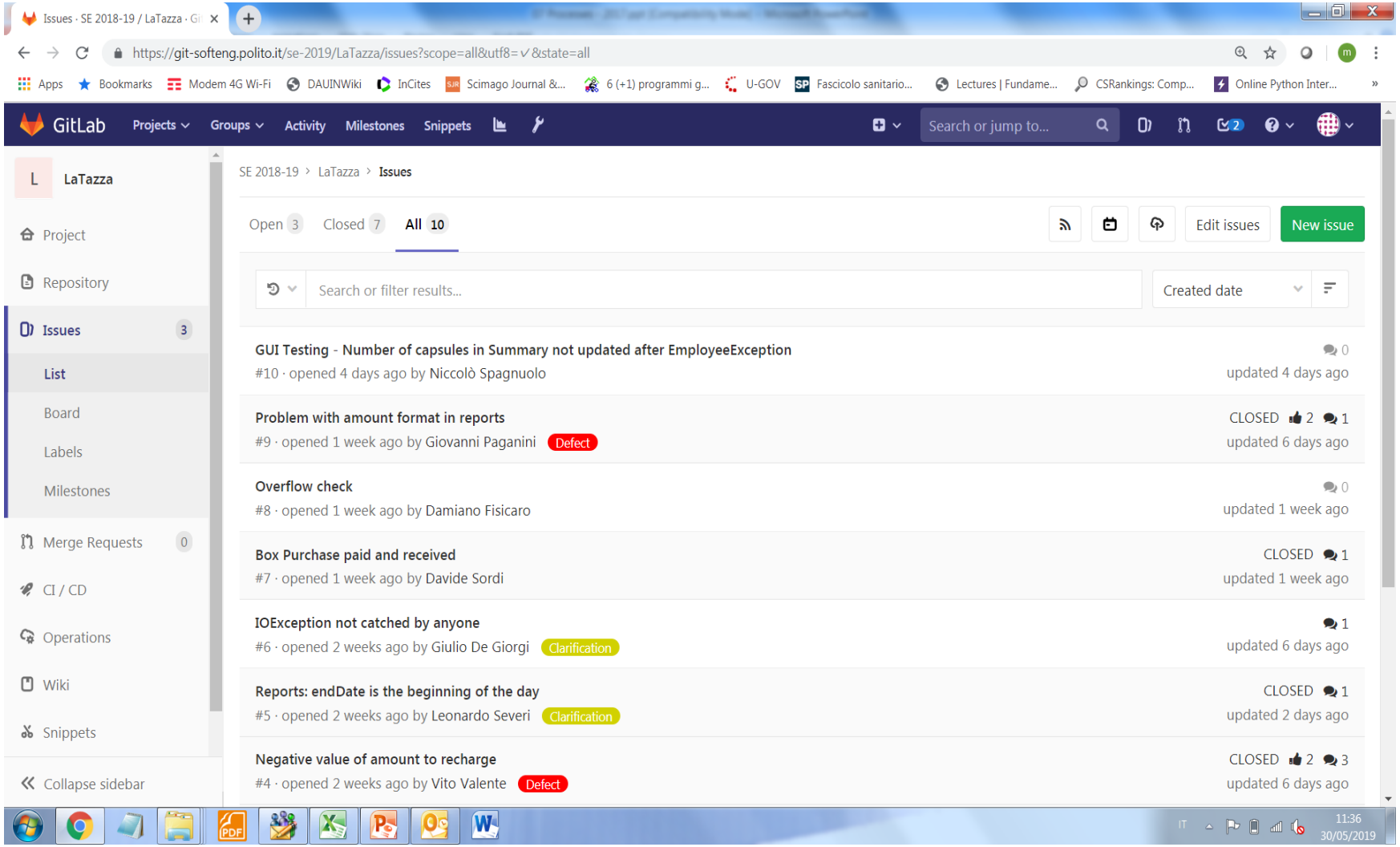
Roles: CR ranking and allocation

- A board (including product architect, market analyst, quality responsible) ranks CRs - vs – project manager does
- A project manager assigns tasks (CRs) to mantainers – vs – maintainer picks CR from list

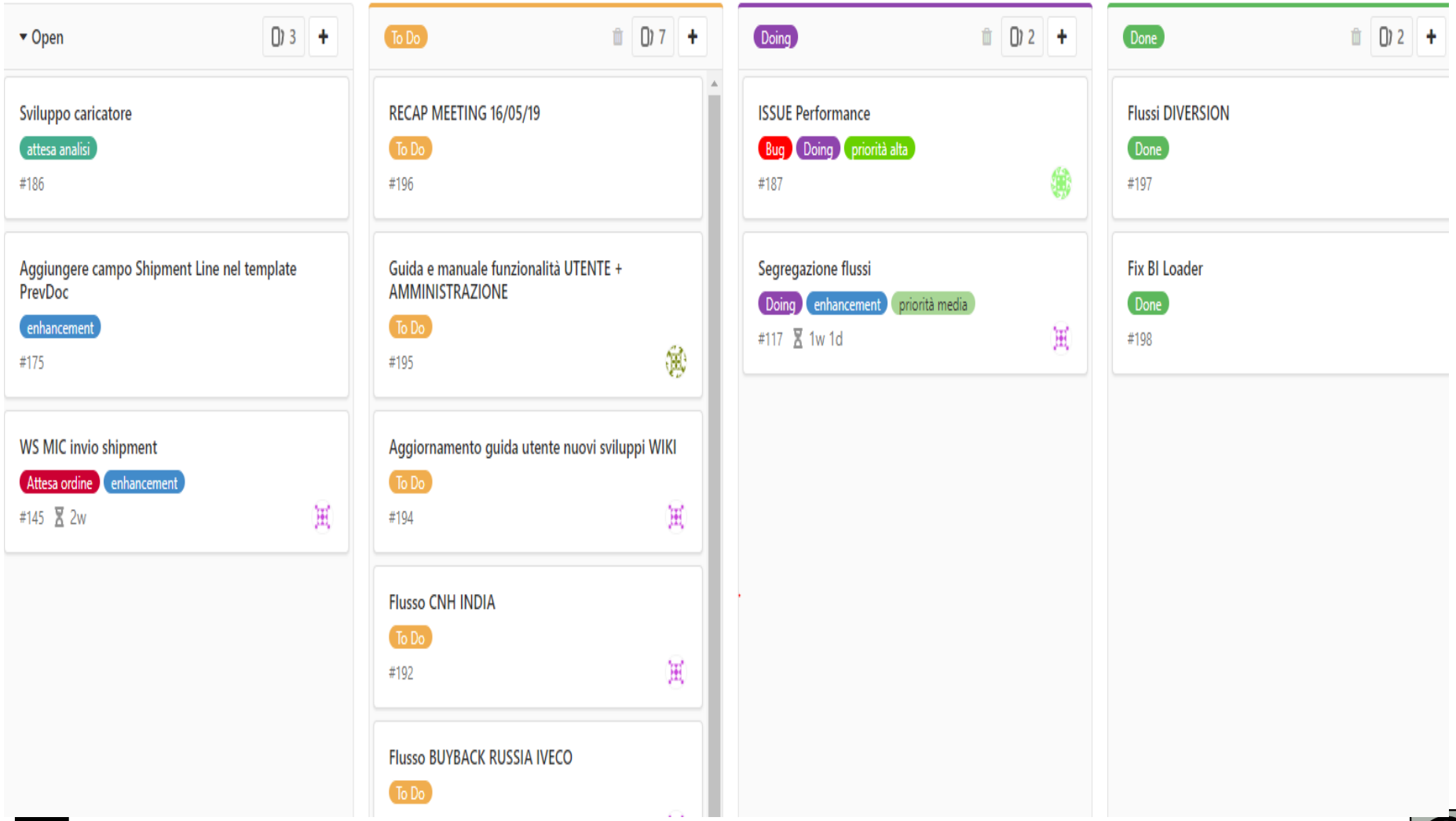
Tools to support change process

- Jira
- Trac
- Redmine
- Pivotal tracker
- Zenhub
- Gitlab - issues
- Bugzilla

Gitlab - Issues



Gitlab issues + categories and labels



Categories and labels

- Categories
  - Open
  - To do
  - Doing
  - Done
- Labels
  - Defect
  - Enhancement
  - Evolution
  - …

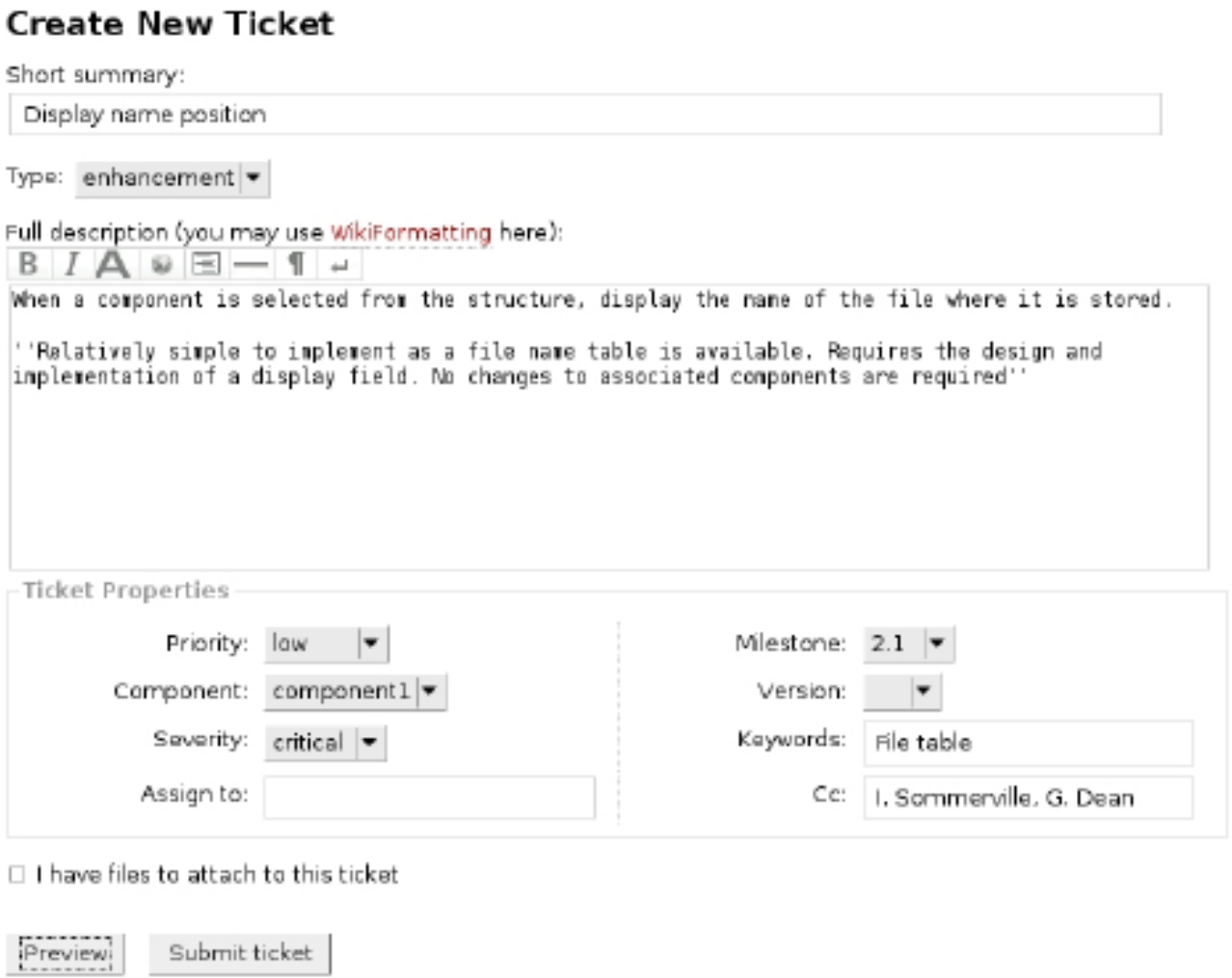
Ex Trac

See Trac demo

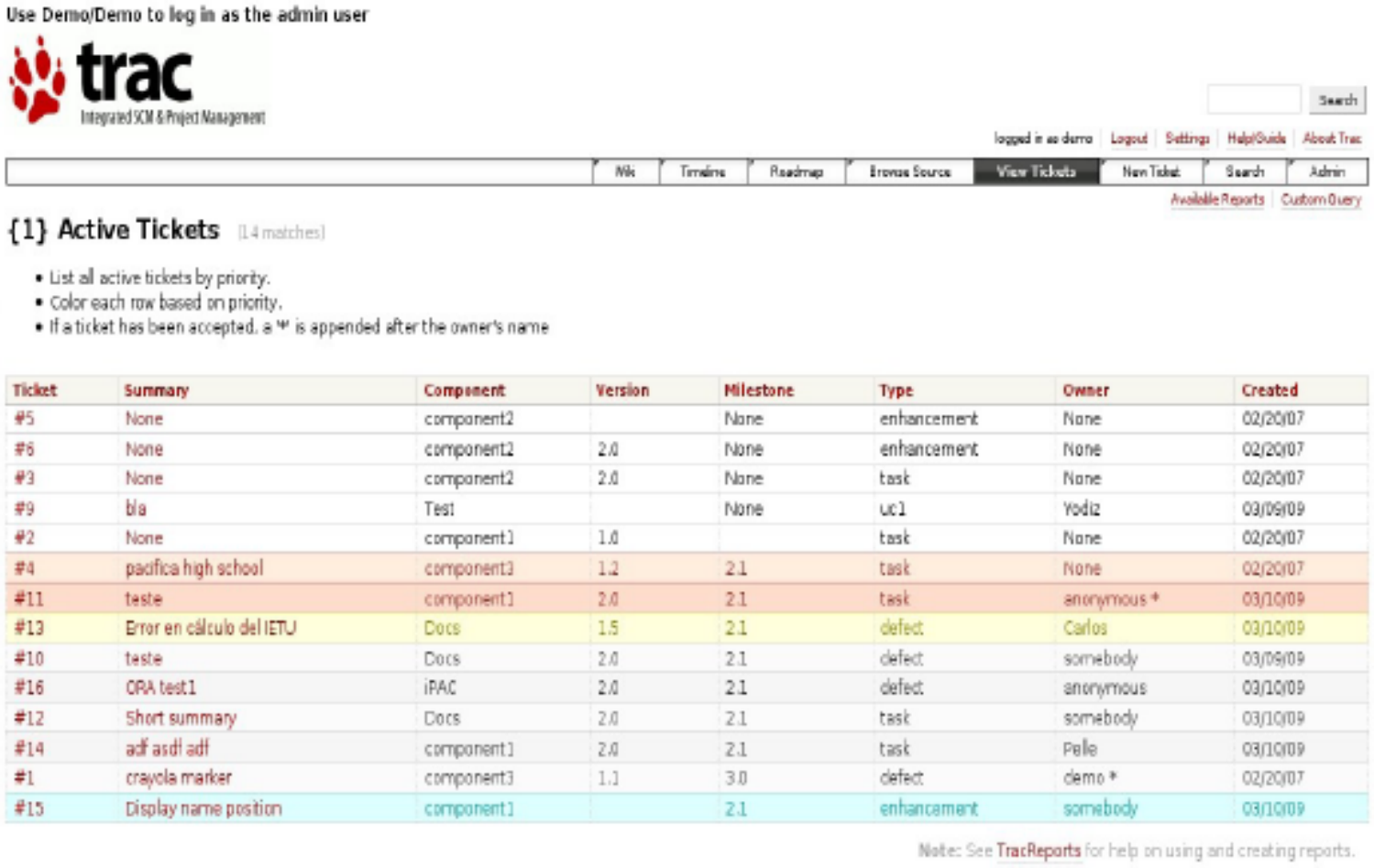
- For trac: change → ticket
- Usr demo pwd demo

- http://www.hosted-projects.com/trac/TracDemo/Demo

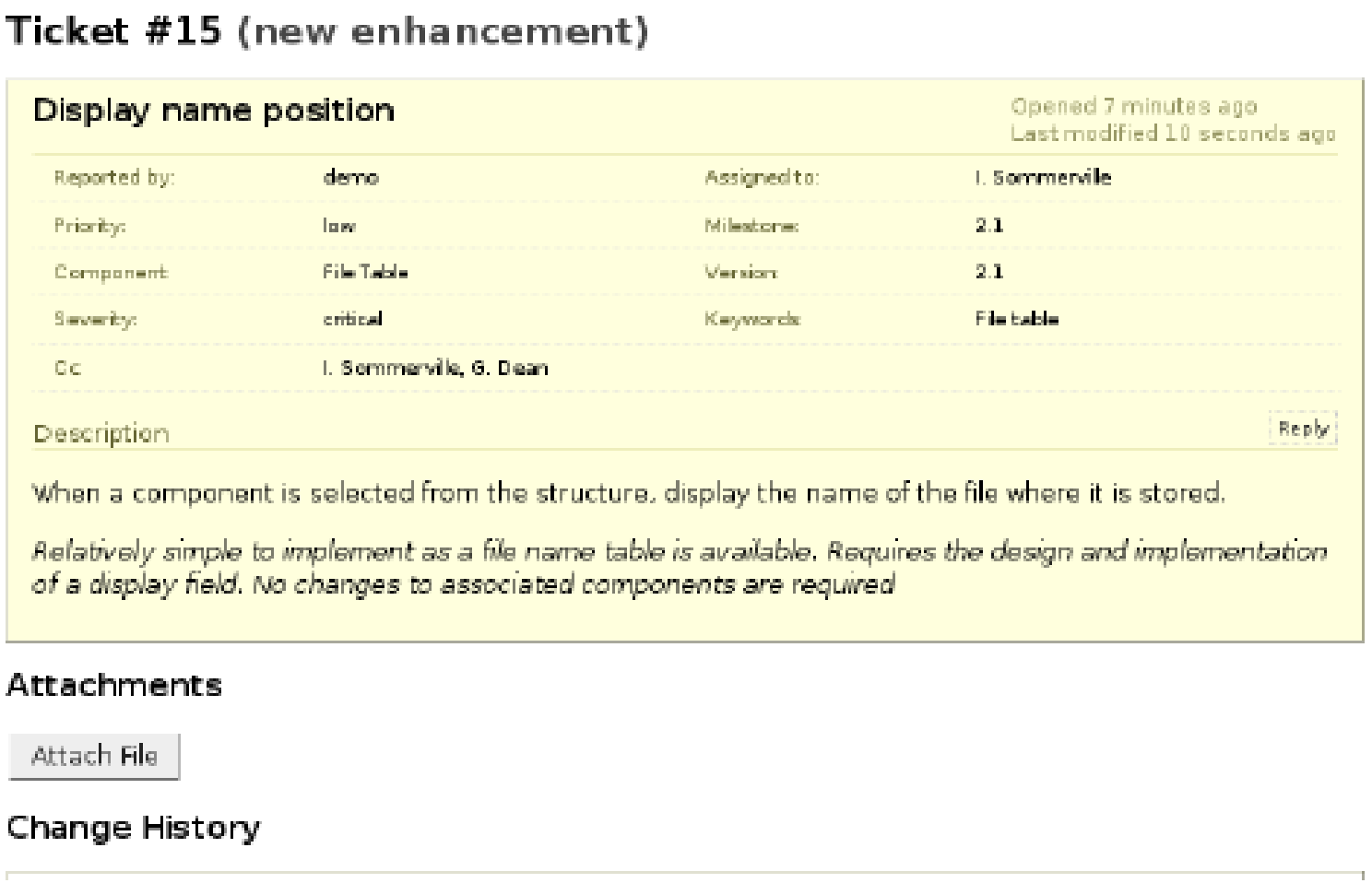
Trac – create ticket



Trac – see all (active) tickets



Trac – open ticket



Ticket #15 (new enhancement)

Display name position

Opened 7 minutes ago
Last modified 10 seconds ago

Reported by:	demo	Assigned to:	I. Sommerville
Priority:	low	Milestone:	2.1
Component:	FileTable	Version:	2.1
Severity:	critical	Keywords:	File table
Cc:	I. Sommerville, G. Dean		

Description — Reply

When a component is selected from the structure, display the name of the file where it is stored.

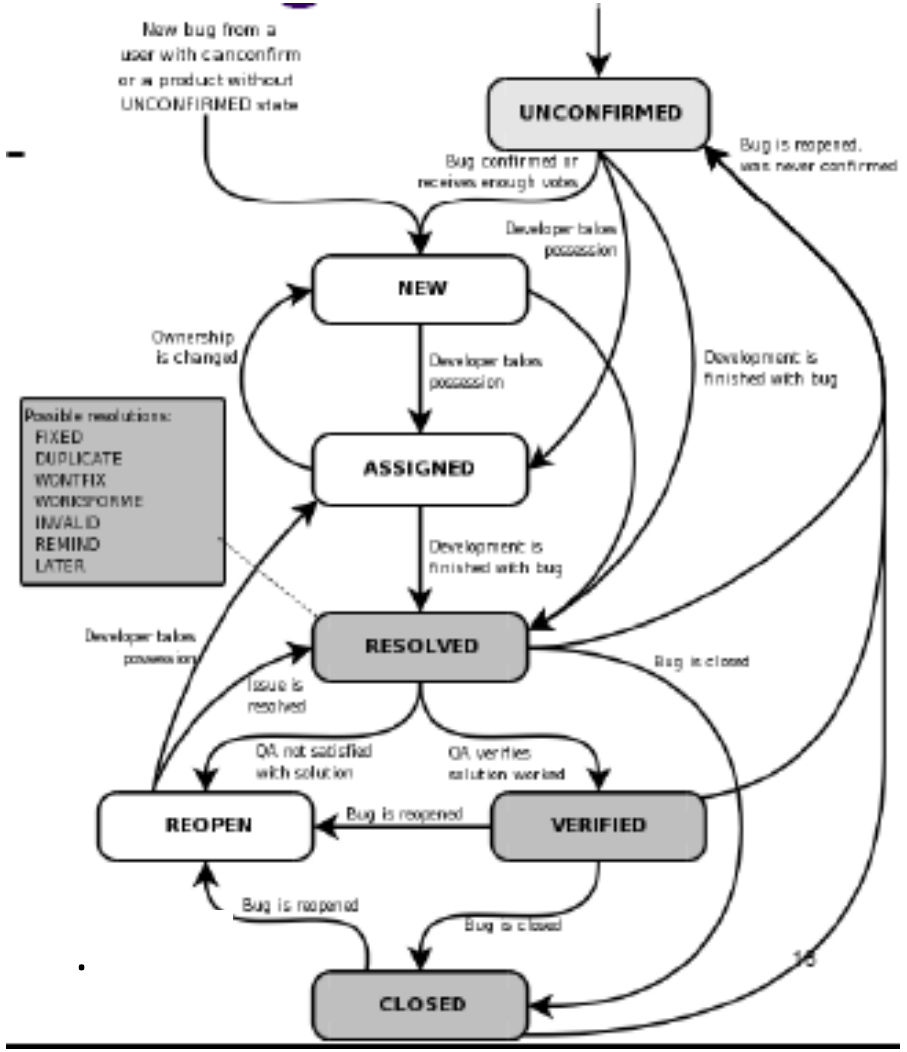
Relatively simple to implement as a file name table is available. Requires the design and implementation of a display field. No changes to associated components are required

Attachments

Attach File

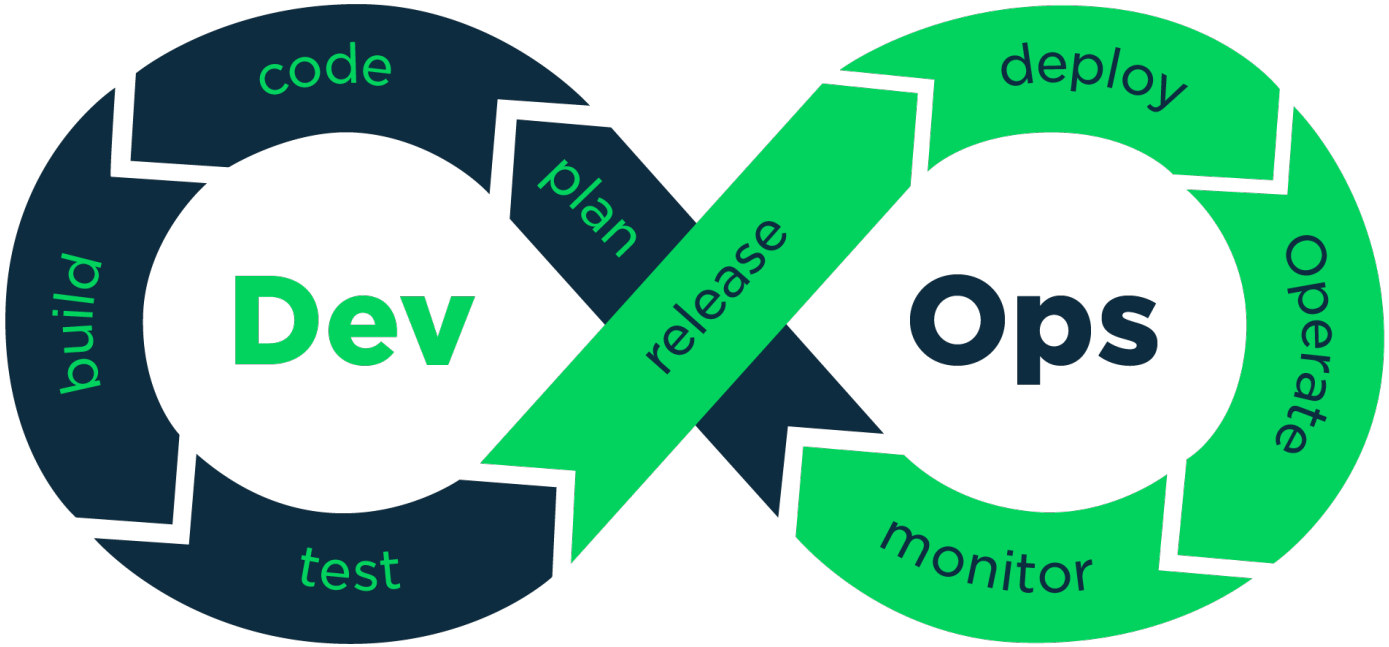
Change History

Lifecycle for change

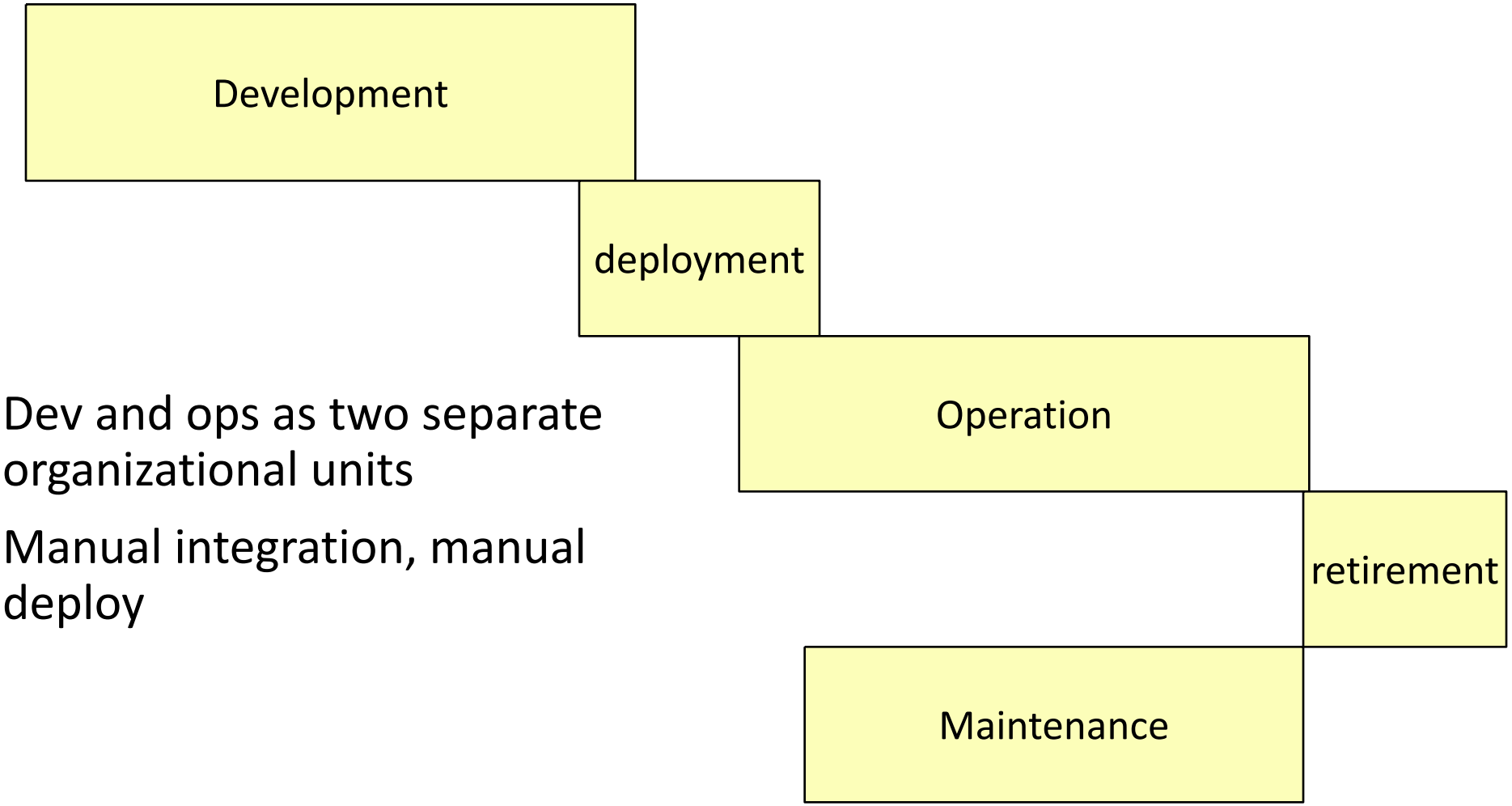


Dev Ops

Remove communication and knowledge barriers between development team and operation team



Before DevOps



Dev and ops as two separate organizational units

Manual integration, manual deploy

Before DevOps - issues

- Knowledge about the project not transferred from dev to ops
- Long delay to evolve the product

DevOps

- Culture / organization
  - Remove communication and knowledge barriers between development team and operation team

- Practices
  - Automated build, automated deployment, automated test
  - CI CD (Continuous Integration, Continuous Deployment)
  - Microservices
  - Infrastructure as code

Example – before devops

- Ecommerce for a shoe company
  - Manual deployment
  - Few hours to deploy (with web site down)
  - Could be done once per week

Example – after devops

- Automated deploy
  - (Octopus, Jenkins..)

- No site down
- Many deployments per day possible

Real cases

Ferrari (racing division)

Software Products

- Electronic Control Unit ECU (embedded sw)
  - power unit: engine, energy recovery (KERS)
  - gearbox
  - brakes
- Support tools
  - simulation (of car)
  - telemetry
  - configuration (30K parameters)
  - pit stop control

Roles

- Mechanical engineer
- Software engineer
  - Application (control theory)
  - Embedded
  - PC

Process and toolchain

- Simulink/stateflow model
- Translation to C
- Compilation to assembly (Freescale)
- Configuration and Tuning
- Upload to firmware

Process and Timing

«The race starts at 2pm»

2 weeks between races

Requirements: 2 days

Coding : 2 days

Test : 4 days

FIA freezing: 3 days

Race: 3 days

In one season (mar – nov)

- A maintenance process
  - 300 versions embedded code
  - 100 versions tools
  - 1000 change requests
    - Few trivial bugs
    - Tens of conceptual defects (requirement misunderstanding)

Key issues

- Tight schedule
- Interaction mechanical engineers- sw engineers
  - Understanding mechanical / control issues
  - Communicating

Apache (and Linux, Mozilla, ..)

Process

- Organized as an evolution/maintenance process
- Time span: many years (10+)

- very successful

Tools and products

- Tools
  - GitHub from 2019
  - Mailing list
  - Bugzilla (Bug tracking)

- Products
  - Source code
  - Test cases
  - (mails, bugs, comments)

Roles and activities

- [Mockus 2000]
  - Core team (2-8 people)
    - Architecture, requirements, integration/build, release
  - Patch developers (10-100)
    - Patch (evolutive + corrective)
  - Bug providers (100 – 1000+)
    - Signal bugs
  - Others
    - Download and use

Facts

- Limited documentation
  - no project plan, no quality plan, no requirement document
- Key points
  - Strict configuration management
  - Simple effective tools for bug/change tracking
  - Hierarchy of roles and responsibilities

  - Top notch, motivated developers (especially in core team)

Lucent

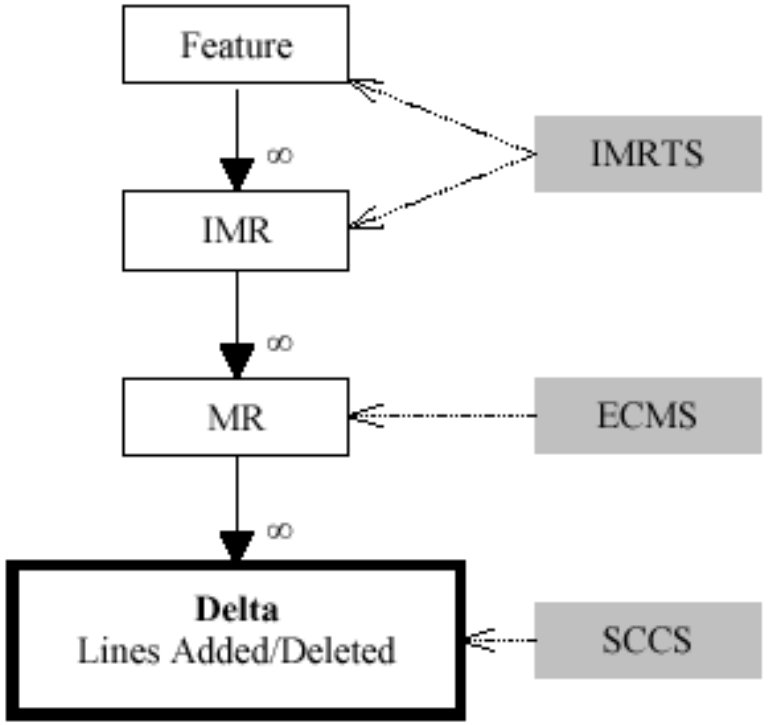
- Major telecom network producer
- Then Alcatel Lucent
- Then Nokia networks

Evolution – 5ESS change process

- 5ESS telephone switching system, Lucent
- 100MLoc, C, C++
- 50 subsystems
- 20 years
- 200 developers (peak) to 50

Change hierarchy

- Feature, composed of (many)
- IMR, initial modification request (logical), Implemented by (many)
- MR (functionally independent, one developer)



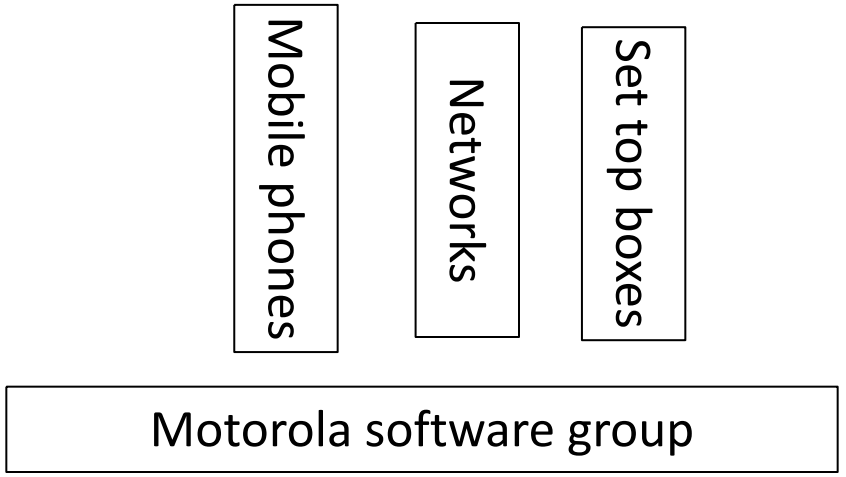
Tools

- Features and IMRs: IMRTS
  - Description
- MRs: ECMS tool
  - Parent IMR, date, files affected, rationale for change
- Configuration management (deltas): SCCS

Motorola GSG

Motorola software group (<2011)

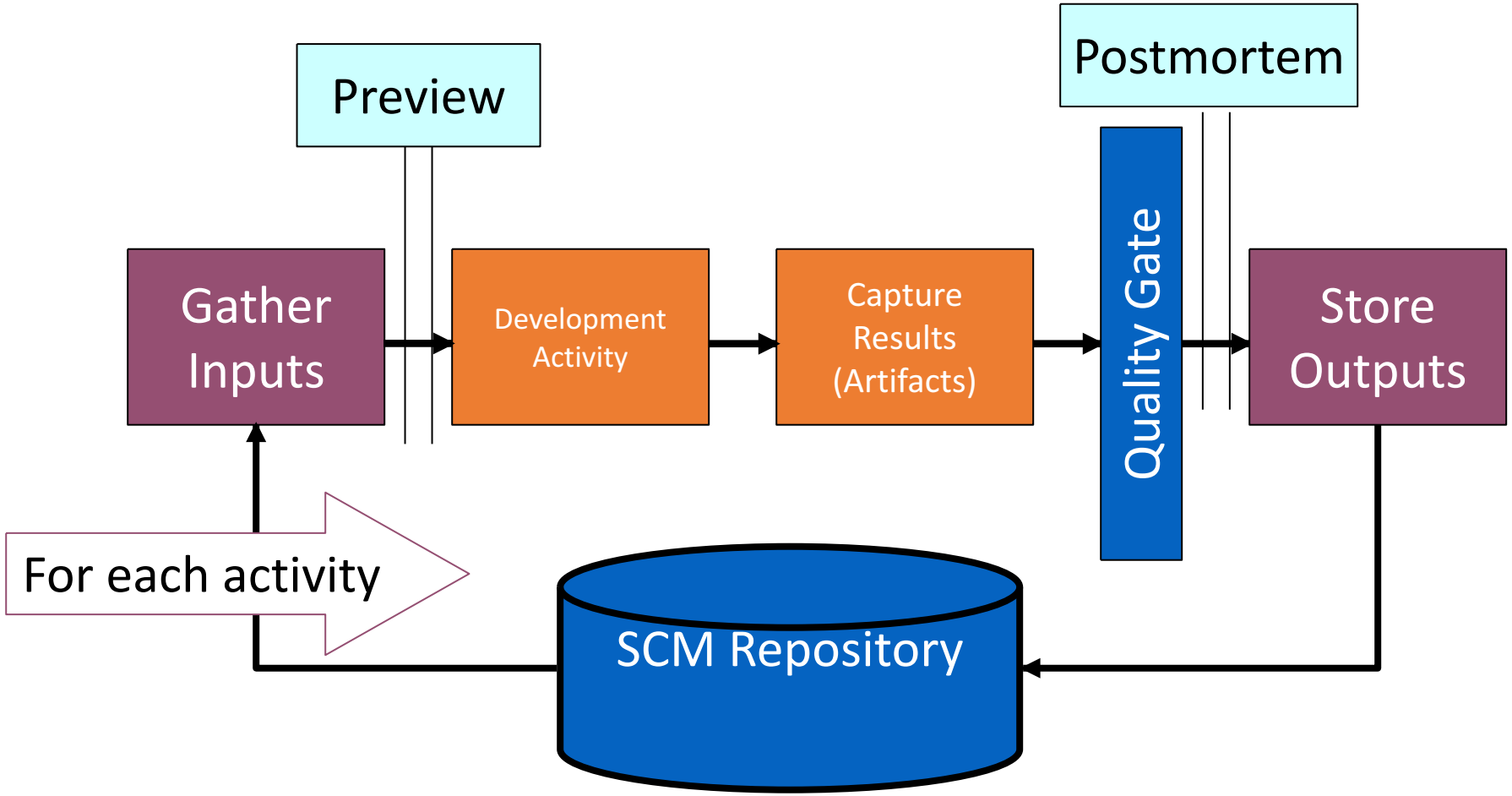
- Horizontal support (software development) to Motorola business units
- Multisite (16 worldwide)



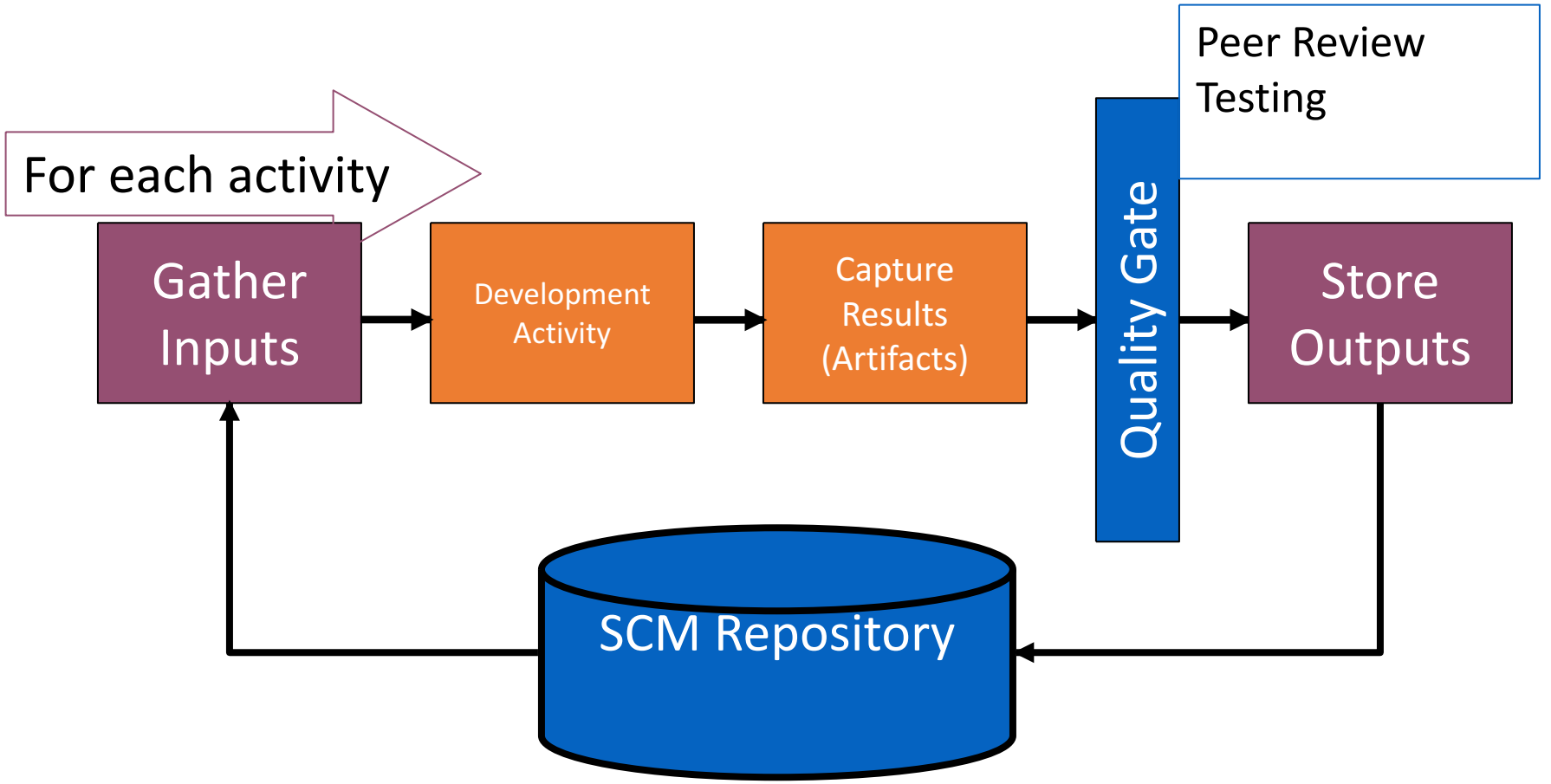
Principles

- CMMI framework
  - Goal: All centers level 5
- Inviolates
  - Project Planning & Tracking
  - Process Framework
  - Previews and Post Mortems
  - Records and Metrics
  - Quality Control (Review & Test)
  - Configuration Management

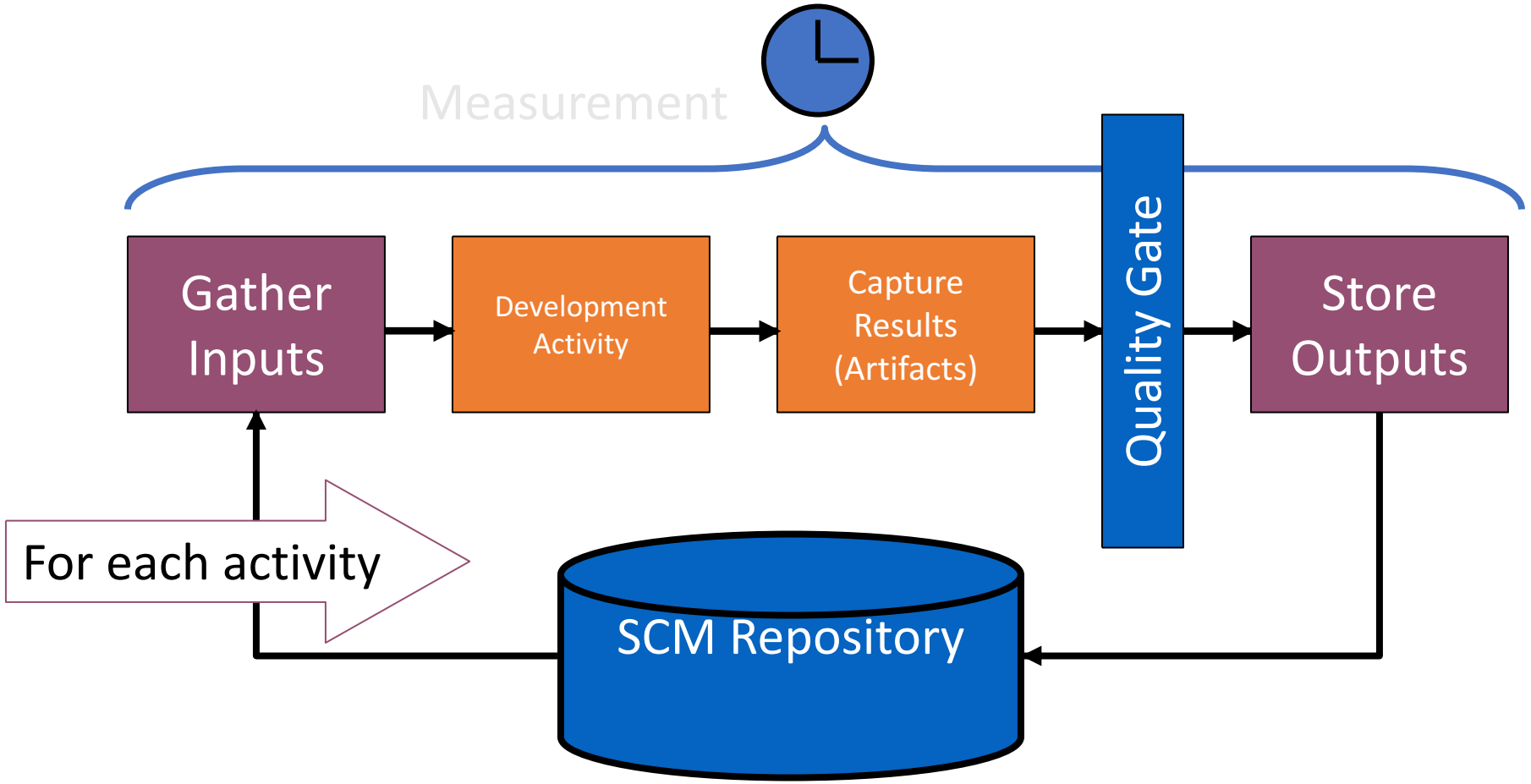
Inviolate: Process framework



Inviolate: Quality control



Inviolate: record and metrics



Synch and Stabilize

- Microsoft, 93-95

Context

- Time to market essential
  - Requirements cant be fixed early on
- Complex products (MLocs), several interacting components,
  - design hard to devise and freeze early on

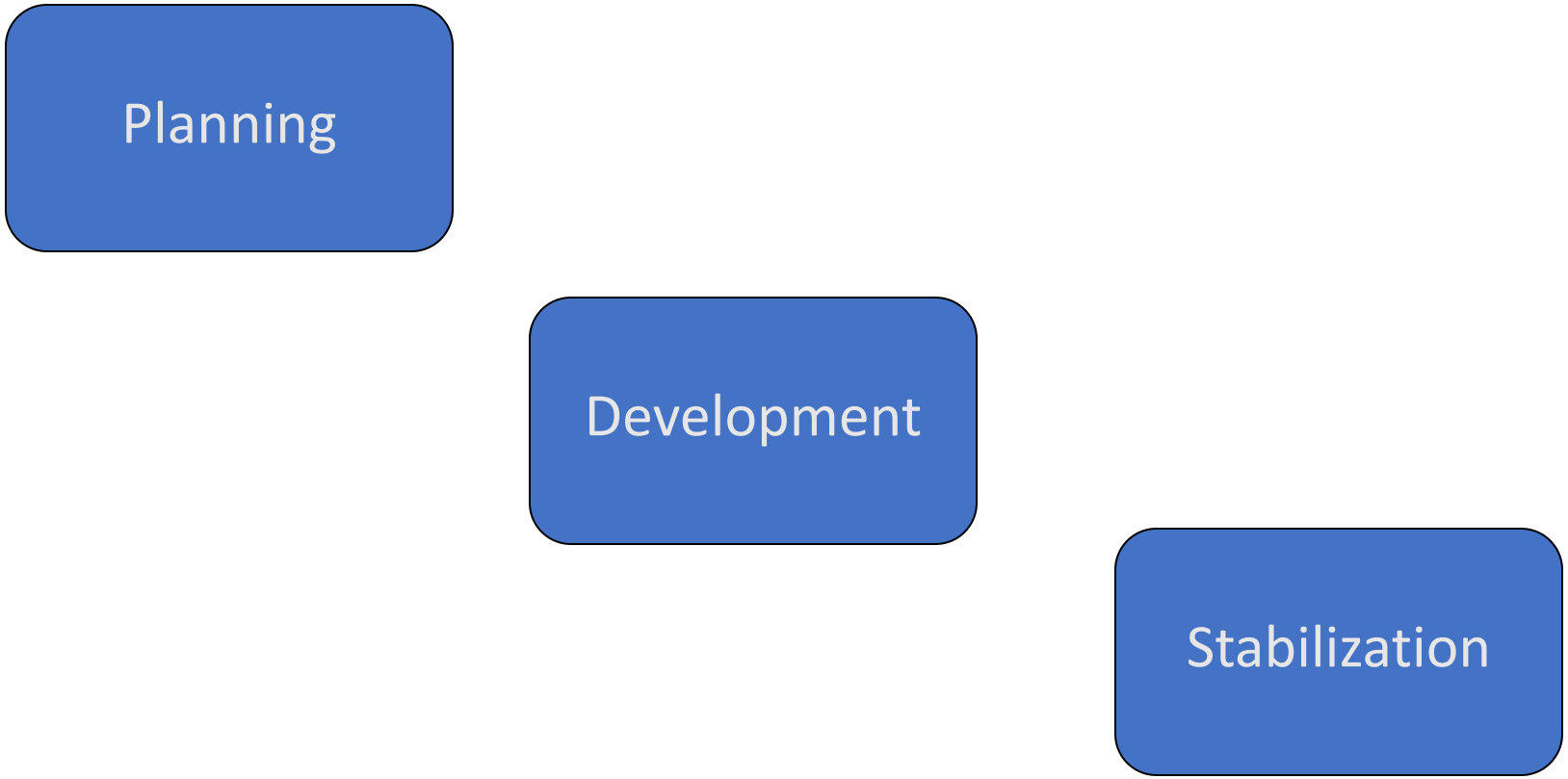
Approach

- Iterations
  - changes to design and requirements
- Small teams (3-8 people) working in parallel
- Maintain hacker culture
- Synchronize frequently
- 1 tester : 1 developer

Synch-and-Stabilize

- Continually synchronize parallel teams
- Periodically stabilize the product in increments versus once at the end

3 Phases



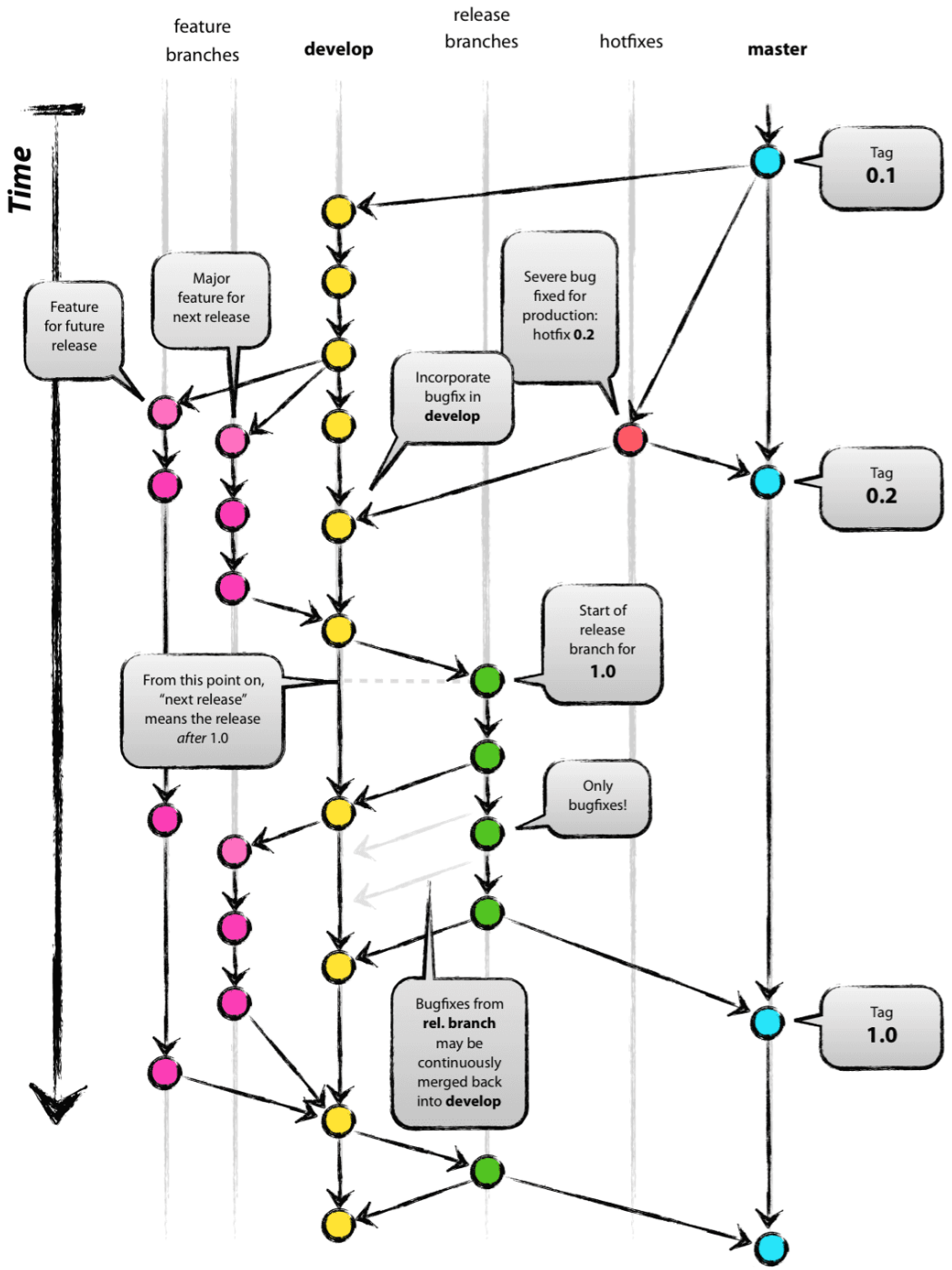
Planning Phase

- Vision Statement - Product Managers
  - Define goals for the new product
  - Priority-order user activities that need to be supported by product features
- Deliverables:
  - Specification document
  - Schedule and “feature team” formation
    - 1 program manager
    - 3-8 developers
    - 3-8 testers (1:1 ratio with developers)

Development

Development Phase

- Plan 3-4 subprojects (lasting 2-4 months each) (iterations)
- Buffer time between iterations
- Each subproject many feature teams
- Subprojects -- design, code, debug
  - starting with most critical features and shared components
  - feature set may change by 30% or more


feature branches
develop
release branches
hotfixes
master
Time
Feature for future release
Major feature for next release
Severe bug fixed for production: hotfix 0.2
Incorporate bugfix in develop
Tag 0.1
Tag 0.2
Start of release branch for 1.0
From this point on, "next release" means the release after 1.0
Only bugfixes!
Bugfixes from rel. branch may be continuously merged back into develop
Tag 1.0

Subproject Development

- Feature teams go through the complete cycle of development, feature integration, testing and fixing problems
- Testers are paired with developers
- Feature teams synchronize work by building the product, finding and fixing errors on a daily and weekly basis
- At the end of a subproject, the product is stabilized

Stabilization

Stabilization Phase

- Internal testing of complete product
- External testing
  - beta sites
  - ISVs
  - OEMs
  - end users
- Release preparation

Five Principles

1. Divide large projects into multiple cycles with buffer time (20-50%)
2. Use a “vision statement” and outline feature specifications to guide projects
3. Base feature selection and priority order on user activities and data
4. Evolve a modular and horizontal design architecture
5. Control by individual commitments to small tasks and fixed project resources

+ Five Principles

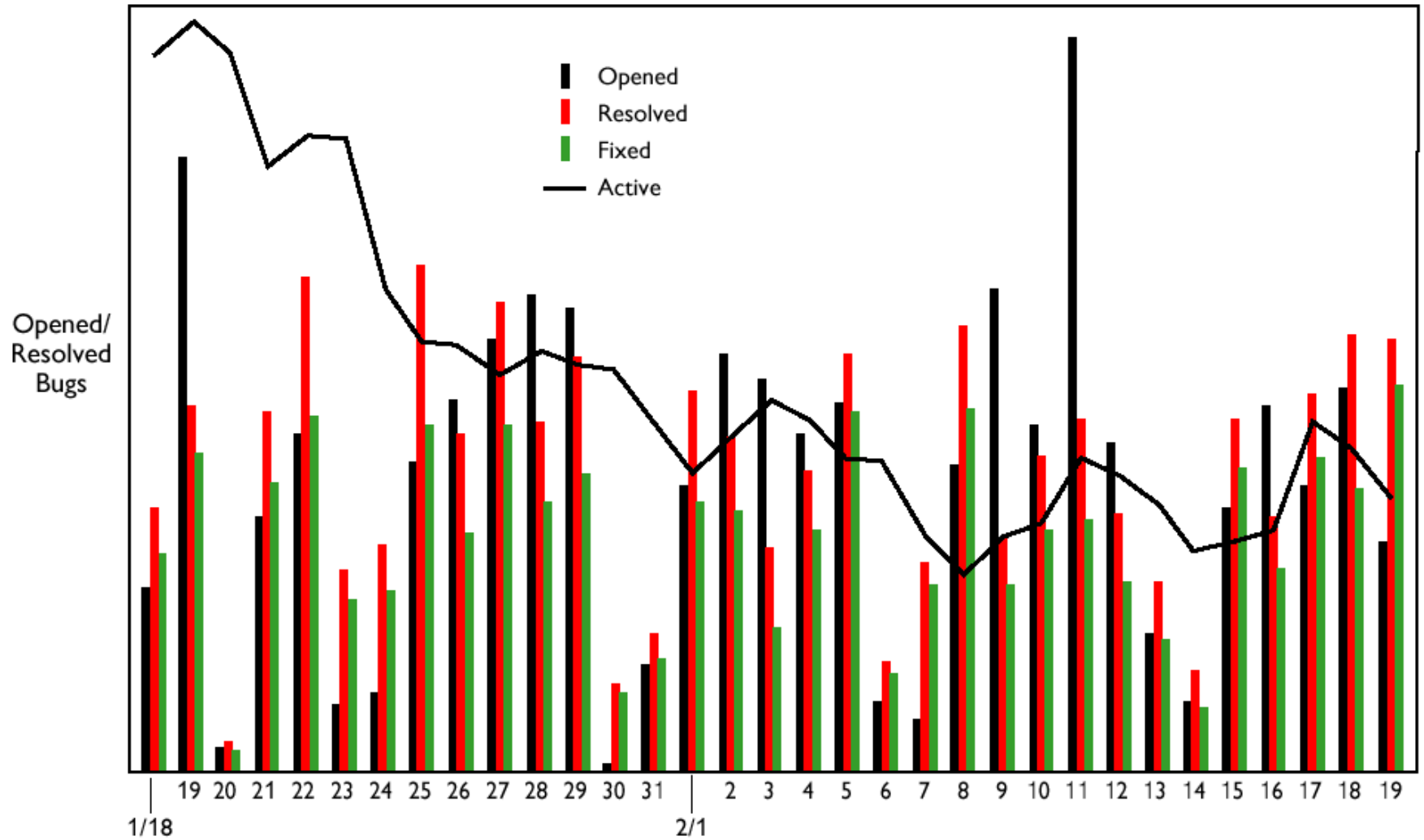
1. Work in parallel teams but “synch up” and debug daily
2. Always have a product you can ship, with versions for every major platform and market
3. Speak a “common language” on a single development site
4. Continuously test the product as you build it
5. Use metric data to determine milestone completion and product release

Rules for Coordination

- Specific time to “check in” code
  - new build every day
  - immediate testing and debugging
- Code that “breaks” the build must be fixed immediately
- Daily builds generated for each platform and for each market

Communication

- All developers at a single physical site
- Common development languages (C/C++)
- Standardized development tools
- Small set of quantitative metrics to decide when to move forward in a project
  - e.g. daily build progress is tracked by the number of new, resolved and active bugs


Opened
Resolved
Fixed
Active
Opened/
Resolved
Bugs
19 20 21 22 23 24 25 26 27 28 29 30 31 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
1/18
2/1

The “Structured Hacker” Approach

- Retain hacker culture
- Add enough structure to make products reliable enough, powerful enough, and simple enough
- Support a competitive strategy of introducing products that are “good enough”, and improving them by incrementally evolving features

Comparison to Traditional

- Waterfall approach “freezes” product specification, then all components are designed, built and merged.
- Prevents
  - changing specifications
  - getting feedback from customers
  - continually testing components as the product evolves

Advantages

- Lends itself to
  - shipping preliminary versions
  - adding features or in subsequent releases
  - easier integration of pieces of products that may still be years away

Synch-&-Stabilize vs. Sequential

- Parallel development & testing
- Vision statement and evolving specification
- Features prioritized in subprojects
- Daily builds (synch), intermediate stabilizations

- Sequential development & testing
- Frozen specification
- All features built simultaneously
- One late, large integration and test phase at the end

Synch-&-Stabilize vs Sequential

- Fixed, multiple release & ship dates
- Continuous customer feedback in development process
- Large teams work like small teams
- Aiming for perfection in each cycle
- Feedback after development as input for next project
- Individuals in separate functional departments work as a large group

Weaknesses

- Microsoft needs more concentrated focus on its product architectures
- Microsoft needs a more rigorous approach to design & code reviews
- S-&-S process may not be suitable for all new products
  - e.g., Video on demand components have real-time constraints that require precise mathematical models
- Does not focus on defect prevention

Benefits

- Breaks down large projects into manageable pieces (priority-ordered sets of features)
- Projects proceed systematically even when it’s impossible to complete a stable design at the outset
- Large teams work like small teams

More Benefits

- Provides a mechanism for customer inputs, setting priorities, completing most important parts first
- Allows responsiveness to the marketplace by “always” having a product ready to ship and having an accurate assessment of which features are completed

Cyber physical ecosystems (wikipedia, OSM, Google, X..)

The ‘city’ model [Kazman 2014]

- No clear boundary
  - APIs and mash ups (ex API for Google Maps)
- Peer content / code production
  - Requirements never known, emerge from individual contributions
- Continuous evolution
  - No releases, no planning

- Continuous operation
  - No development – maintenance
  - Always on and always changing
- Open teams, unstable resources
- Sufficient correctness
  - Always ‘beta’
- Emergent behaviours
  - Unplanned behaviours and functions
  - (Twitter as coordination tool of masses)

The structure

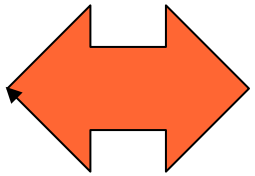
- Core/Kernel
  - Horizontal functions
  - Always on, top reliability, closed, slow to change
- Periphery
  - End user functions and content
  - Fast changing, open

Bifurcation

- Core
  - Core requirements / architecture well defined
  - Controlled by core team
  - Reliability: high
- Periphery
  - Unclear, instable req and design
  - No control, crowdsourcing
  - Reliability: variable

Process selection

Process selection

Product  **Process**

Process attributes

- Sequential vs parallel activities
  - (i.e. documents can be modified in parallel or not)
- Iterations vs no iterations
  - Duration of iteration
- Time framed vs not
- Colocation of staff
- Emphasis on documents (need for certification, compliance)

Process comparison

	Agile	26262	RUP	Synch Stab
Iteration number	many	One+	few	few
Iteration duration	weeks	long	months	months
Parallel activities	yes	no	yes	Yes
Documents	few	many	many	few
Colocation staff	yes	Yes or not	-	Yes

Product attributes

- Criticality
- Size
- Domain
- Lifespan
- Lifecycle phase
- Bespoke / market driven
- Ownership
- Relationship user - developer

Criticality

- Criticality
  - Safety critical, mission critical, other
  - More criticality, more emphasis on: reliability, safety, security
    - Norms and laws applied, legal responsibility issues
    - ex ISO IEC 61508 for safety critical functions
    - Ex Iso 26262 for safety critical functions in automotive

Criticality

- Safety critical
  - Typically waterfall
    - (sequential activities, documents for compliance)

Size

- Size
  - LOCs (Lines of Code)
  - Duration of development, team size
  - Number of subcontractors

  - Effect on coordination and communication during development and maintenance

Size

- Large
  - Sequential activities
  - Documents for coordination of teams
  - Longer iterations

  - Waterfall or modified waterfall (synch and stabilize, RUP)

Domain

- Aerospace, medical, automotive, industry, banking, insurance, ...
  - Effect on norms and laws applied:
    - ex Basel 3 in finance, 26262 in automotive, Do178B in aerospace
  - Effect on responsibility of developer, operator, maintainer

Domain

- Domain often correlates with safety critical
  - Automotive, aerospace, medical, process industry (chemicals, nuclear)
  - And therefore, points to waterfall standards

Lifespan

- Years
  - Maintenance is the leading cost center
    - Maintainability is key concern
    - Cost of non quality is key concern
- Months
  - Development is the leading cost center
    - Maintainability less important
  - Remember: cost of quality vs cost of non quality - or technical debt

Lifecycle phase

- New development
  - Waterfall, rup, agile
- Maintenance
  - Cyclic management of change requests, effects of legacy

Bespoke / market driven

- Bespoke: one customer / end user
  - ‘simpler’ requirement collection
  - ‘simpler’ architecture
    - must satisfy one case only
- Market driven: many customers / end users
  - A variety of customers / users
    - Harder requirement definition
    - Architecture must encompass wider variability
  - Higher competition and stringent time to market

- Effect on time to market, requirement collection and ranking, cost structure and business model

Ownership

- Full property
  - Code modification possible
- Copyright
  - No code modification
  - Mechanisms for parameterization/adaptation
- Copyleft
  - Code modification possible, but costly

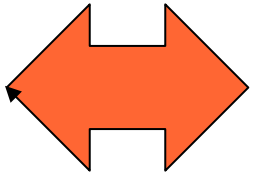
Relationship with developer

- Internal department of company
  - Looser definition of requirements
- External company
  - Need to formalize requirements and contract

Process selection

Product attributes

- Criticality
- Size
- Lifetime
- Bespoke /market driven



Process attributes

- Parallel activities
- Iterations
- Time framed
- Colocation of staff
- Documents based

Process selection

- Understand product attributes
- Rank –ilities
- Apply rules of thumb

Rules of thumb

- Reliability, safety
  - Waterfall –like (document-based, few iterations, no parallel activities)
- Market driven (Time to market)
  - Frequent iterations (agile like)
- Colocation of staff
  - No: document-based, no parallel activities (agile like)

Rules of thumb 2

- Size
  - The bigger, the more documents and activities
    - Waterfall like
- Lifetime
  - Long: documents
    - Waterfall like

Summary

- There are many options for organizing the basic activities
  - Requirements, design, coding, testing

  Responsibilities, roles

  → process models

Summary

- Process models can be characterized in terms of
  - New development / maintenance
  - Iterations, parallel activities, documents, location of staff

Summary

- Process models
  - New developements
    - Waterfall, V-Model, RUP
    - Agile (XP, Scrum)
  - Maintenance
    - Change based
- Actual processes are inspired by the models
  - Examples: Motorola, Ferrari, Lucent, Apache, ..

Summary

- Projects can be characterized by
  - Domain / criticality
  - Size
  - Lifetime
  - Bespoke / mass market

- The process should be selected/created in function of the project

Relationship developer / user

Typical scenario

1 Bespoke, external, property, *new*

- Company (ex Polito) needs custom software product (ex Polito APP)
- Company SW develops it
  - Inception, for requirement analysis and contract negotiation
  - Contract signature
  - Development, delivery

Typical scenario

2 Bespoke, external, property, *maintenance*

- Company (ex Polito) needs maintenance work on owned software product P (Ex POLITO App)
- Company SW performs maintenance work
  - Similar to case 1, but typically contract is per year and involves a fixed amount of work (or dedicated staff) in the year (ex 400 p-days) and not an amount of functionality (as in scenario 1)

Typical scenarios

1-a, 2-a

same as above, but internal

- typical of bank, insurance,
- internal IT department instead of external vendor
- No legal contract, but similar negociation of functionality / effort requested between user department and IT department

Typical scenarios

3 COTS, external, copyright

- Company (ex Microsoft) develops and sells a mass market product (ex Windows)

  - Fixes and patches on the current release (maintenance thread)
  - Work on next release (new development thread)